

BACHELOR OF SCIENCE IN COMPUTER SCIENCE AND ENGINEERING

**Orthogonal Defect Classification on Defects4J Using an LLM-Driven
Scientific Approach**

Md. Sakib Hossain

210042133

Mohammad Nahiyan Kabir

210042168

Hasin Mahtab Alvee

210042174

Department of Computer Science and Engineering

Islamic University of Technology

September, 2026

**Orthogonal Defect Classification on Defects4J Using an LLM-Driven
Scientific Approach**

Md. Sakib Hossain

210042133

Mohammad Nahiyen Kabir

210042168

Hasin Mahtab Alvee

210042174

Department of Computer Science and Engineering

Islamic University of Technology

September, 2026

Declaration of Candidate

This is to certify that the work presented in this thesis is the outcome of the analysis and experiments carried out by **Md. Sakib Hossain, Mohammad Nahiyan Kabir,** and **Hasin Mahtab Alvee** under the supervision of **Lutfun Nahar Lota**, Assistant Professor, Department of Computer Science and Engineering and co-supervision of **Ishmam Tashdeed**, Lecturer, Department of Computer Science and Engineering, Islamic University of Technology, Dhaka, Bangladesh. It is also declared that neither this thesis nor any part of it has been submitted anywhere else for any degree or diploma. Information derived from the published and unpublished work of others have been acknowledged in the text and a list of references is given.

Lutfun Nahar Lota

Assistant Professor

Department of Computer Science and Engineering

Islamic University of Technology (IUT)

Date: September 17, 2026

Md. Sakib Hossain

Student ID: 210042133

Date: September 17, 2026

**Mohammad Nahiyan
Kabir**

Student ID: 210042168

Date: September 17, 2026

Ishmam Tashdeed

Lecturer

Department of Computer Science and Engineering

Islamic University of Technology (IUT)

Date: September 17, 2026

Hasin Mahtab Alvee

Student ID: 210042174

Date: September 17, 2026

Dedicated to our families, friends, and mentors

Contents

1	Introduction	1
1.1	Defects4J: The Benchmark Dataset	1
1.2	Motivation	2
1.3	Problem Statement	4
1.3.1	Limitations of Existing Approaches	4
1.3.2	The Research Gap	5
1.3.3	Problem Definition	5
1.4	Research Objectives	6
1.5	Research Questions	6
1.6	Contributions	7
1.7	Report Organization	7
2	Related Works	9
2.1	Existing Defects4J Bug Type Classification	9
2.2	LLM-based Defect Classification	10
2.3	The Taxonomy Problem	11
2.4	Orthogonal Defect Classification (ODC)	12
2.4.1	Impact, and Why This Study Does Not Classify It	14
2.5	Human Agreement on ODC Classification	15
2.6	Summary	16
3	Proposed Methodology	17
3.1	Pipeline Architecture Overview	17
3.2	ODC and Defects4J Artifact Mapping	18
3.3	Evidence Collection	20
3.4	Evidence Modes: Pre-fix versus Post-fix	22
3.5	LLM-based ODC Classification	22
3.5.1	Taxonomy Axis: How Strict the Vocabulary Is	22

3.5.2	Strategy Axis: How the Model Reasons	23
3.5.3	Valid Conditions	23
3.6	Prompt Design	24
3.7	Scientific Debugging Protocol	25
3.7.1	An Example Turn	27
3.8	Classification Pipeline and Validation	28
3.9	Evaluation Framework	28
3.9.1	The Reference Standard	29
3.9.2	The Four-Tier Framework	29
3.9.3	Reading Tiers 2 and 3	30
3.10	Summary	31
4	Experimental Design and Data Collection	32
4.1	Dataset: Defects4J	32
4.2	Experimental Setup	32
4.2.1	Evidence Collection Configuration	32
4.2.2	LLM Configuration	33
4.2.3	Evaluation Configuration	34
4.3	Data Collection	36
4.4	Multi-Fault Context	37
5	Results and Analysis	38
5.1	Classification Results Overview	38
5.2	RQ1: Defect-type Distribution	39
5.2.1	Results	39
5.2.2	Statistical Analysis	40
5.2.3	Discussion	40
5.3	RQ2: Taxonomy Coverage	41
5.3.1	Results	41
5.3.2	Discussion	42
5.4	RQ3: Classification Accuracy	42
5.4.1	Results	42
5.4.2	Statistical Analysis	44
5.4.3	Label Changes When the Fix Is Visible	44
5.4.4	Discussion	45
5.5	RQ4: Contribution of Each Component	45
5.5.1	RQ4a: Taxonomy Grounding	46
5.5.2	RQ4b: The Enforced Reasoning Loop	47

5.6	Why Types Diverge	49
5.7	Qualitative Analysis of Thirteen Bugs	50
5.7.1	S1: Both Modes Agree and Both Are Correct	51
5.7.2	S2: Both Modes Agree and Both Are Wrong	51
5.7.3	S3: The Fix Leads to a Worse Label	51
5.7.4	S4: The Fix Leads to a Better Label	52
5.7.5	S5: Both Modes Wrong, with Different Labels	52
5.7.6	S6: Scientific and Few-Shot on Identical Evidence	53
5.7.7	Strategies Against the Manual Reading	53
5.7.8	When the Developer Had More Than One Option	54
5.7.9	Findings from the Manual Analysis	56
5.8	Cross-Model Analysis	57
5.8.1	Behaviours of the Pipeline	57
5.8.2	Behaviours of the Model	58
5.9	A Closer Study of the Scientific Loop	59
5.9.1	Where the Loop and the Few-Shot Prompt Differ	60
5.9.2	Where the Loop Does Better	60
5.9.3	Where the Few-Shot Prompt Does Better	61
5.9.4	Why Gemini Stops Early and gpt-5-mini Keeps Investigating	61
5.9.5	Lessons for the Scientific Loop	62
5.10	Summary	63
6	Threats to Validity	64
6.1	Internal Validity	64
6.2	Construct Validity	64
6.3	External Validity	65
6.4	Other Threats	66
7	Conclusion	67
7.1	Summary	67
7.2	Limitations	68
7.3	Future Work	69
7.4	Closing Remarks	69
	References	71

List of Figures

3.1	Overview of the proposed methodology: data collection from Defects4J artifacts into per-bug evidence contexts, ODC classification through the enforced reasoning loop, and evaluation across the research questions	18
3.2	Mapping between Defects4J artifacts and ODC opener/closer attributes. Impact is shown for completeness; this study classifies Defect Type only (Section 2.4.1).	19
3.3	The three classification strategies (rows) crossed with the three vocabulary settings (columns). Five of the nine cells are valid; the four grey cells are invalid combinations, since <i>zero</i> carries no vocabulary constraint and <i>free</i> defines none. <i>scientific-open</i> (bold) is the default for both pre-fix and post-fix evidence.	24
3.4	Prompt template of the <i>few</i> strategy: all guidance and evidence in a single call. Highlighted names are dynamically filled components; text after # is explanatory.	25
3.5	Prompt template of the <i>scientific</i> strategy: the same bug context, but the decision tree and examples are replaced by a turn protocol in which further evidence is revealed only through probes the model requests. .	25
3.6	The enforced reasoning loop, adapted from the AutoSD loop-iteration structure [19]. The failure observation is the prompt input; each turn then runs hypothesis, prediction, experiment, observation, and conclusion.	27
3.7	The four-tier evaluation framework. Tiers 1 to 3 progressively relax the agreement criterion; Tier 4 corrects Tier 1 for chance agreement. . . .	30

List of Tables

2.1	ODC opener and closer attributes [15]	12
2.2	The seven ODC Design/Code defect types [6], [15], grouped by this study into two families	13
2.3	Reported human agreement on ODC-style defect classification, ordered by the evidence available to the raters	15
3.1	The evidence probes available to the scientific strategy	26
4.1	Defects4J v2.0 projects and their approximate active bug counts. Per-project counts are version-dependent.	33
4.2	Research questions, the conditions each uses, and the metrics that answer it	34
4.3	The six projects in the study corpus	36
5.1	The corpus used for all results in this chapter	38
5.2	RQ1: distribution of ODC defect types under the four runs (410 bugs each)	39
5.3	RQ1: per-project type counts, <i>scientific-open</i> pre-fix	40
5.4	RQ1: test of association between project and ODC type (10,000 permutations)	40
5.5	RQ2: taxonomy coverage. The escape category was available in all 1,640 classifications and was never selected.	41
5.6	RQ3: four-tier agreement between the pre-fix classification and the post-fix reference (410 bugs; 95% bootstrap intervals in parentheses)	43
5.7	RQ3: severity of disagreement, conditional on strict match having failed	43
5.8	RQ3: the three most frequent drift pairs, <i>scientific-open</i> . $A \rightarrow B$ counts bugs labelled A in pre-fix mode and B in post-fix mode.	44
5.9	RQ4a: ablation ladder over 410 bugs. “Reproducible” is the share of bugs for which the model produced the identical label in both evidence modes.	46

5.10	RQ4a: free-form labels for four bugs under the <i>zero-free</i> condition. Each pair describes a similar defect, but no pair can be aggregated as the same label.	46
5.11	RQ4b: bugs improved and degraded by the enforced loop, relative to the few-shot prompt	47
5.12	RQ4b: improved and degraded bugs by the pre-fix loop's use of held-back evidence	48
5.13	The thirteen manually analysed bugs. CHK = Checking, ALG = Algorithm/Method, ASN = Assignment/Initialization, INT = Interface/O-O Messages, REL = Relationship, FCO = Function/Class/Object. .	50
5.14	Bugs whose ODC defect type depended on which repair the developer chose. Types in parentheses are those of the manual reading; abbreviations as in Table 5.13.	54
5.15	Labels assigned by both models to the thirteen scenario bugs. S = <i>scientific-open</i> , F = <i>few-open</i> ; labels matching the manual reading are shown in bold. Abbreviations as in Table 5.13.	57
5.16	Agreement with the manual reading and reasoning behaviour of both models on the thirteen scenario bugs	58
5.17	Scenario bugs on which exactly one strategy matches the manual reading, on identical evidence	60
5.18	Summary of findings for the four research questions	63

List of Abbreviations

APR	Automated Program Repair
API	Application Programming Interface
CFG	Control Flow Graph
CI	Confidence Interval
CLI	Command Line Interface
CoT	Chain-of-Thought
D4J	Defects4J
DFG	Data Flow Graph
LLM	Large Language Model
ML	Machine Learning
NLP	Natural Language Processing
ODC	Orthogonal Defect Classification
RQ	Research Question
SDLC	Software Development Life Cycle
SVM	Support Vector Machine
TVD	Total Variation Distance

Acknowledgement

We would like to express our sincere gratitude to our supervisor, Lutfun Nahar Lota, and our co-supervisor, Ishmam Tashdeed, for their continuous guidance and support throughout this research. Their expertise and encouragement helped shape this work from concept to completion.

We are also grateful to the Network and Data Analysis Group (NDAG) Lab at IUT for providing the research environment and resources that made this thesis possible.

Finally, we thank the maintainers of the Defects4J benchmark, whose careful curation of reproducible real-world bugs made this study possible.

Abstract

Defect classification turns isolated bug reports into process-level signals that guide debugging strategy, resource allocation, and quality improvement. Defects4J is the standard empirical benchmark for Java fault studies, yet no prior work has labelled it with a principled, semantic taxonomy. Existing classification efforts cover only tens to a few hundred bugs, rely on structural or patch-shape taxonomies that describe *how* a bug manifests rather than assigning it a semantic defect type, or depend on post-fix code, which is unavailable during real triage.

This thesis presents an automated pipeline that classifies Defects4J bugs into the seven Orthogonal Defect Classification (ODC) Design/Code defect types using Large Language Model (LLM) reasoning. We structure classification as a two-variable experimental design: a *taxonomy* axis, which controls the label space the model may answer in, crossed with a *strategy* axis, which controls how the model reasons toward a label. The strongest strategy is an enforced reasoning loop, inspired by scientific debugging, in which the model commits to a hypothesis and then probes held-back evidence to confirm or refute it. An explicit “Other” escape category lets us measure the coverage of the taxonomy instead of assuming it. We classify every bug in two evidence modes, pre-fix (symptoms only) and post-fix (with the buggy-to-fixed diff), which provides an evaluation reference for a benchmark that carries no ODC labels.

We evaluate the pipeline on 410 bugs from six Defects4J projects, giving 1,640 classifications. Three ODC types account for 97.8% of the bugs, and under the default condition the type distribution does not differ significantly across projects. The “Other” category was available in every classification and was never selected, which bounds the true escape rate at 0.2%. A classification made from pre-fix evidence reproduces the fix-informed reference on 70.2% of bugs (Cohen’s $\kappa = 0.505$), and is unrelated to it on only 2.4%. Compared with a strong few-shot prompt, the enforced loop improves the classification of 73 bugs and degrades 58, with significantly higher chance-corrected agreement, and its gain is concentrated where it requests held-back evidence. Roughly 30% of labels change once the fix becomes visible, but 80% of those changes remain within the same three closely related defect types, and the aggregate type distribution remains almost unchanged. Running the pipeline with a second model, gpt-5-mini, on the manually analysed scenario bugs shows that the pre-fix advantage of the loop and the main failure modes arise from the pipeline, while how long the loop investigates and how it uses the fix diff depend on the model.

Chapter 1

Introduction

Software bugs are an unavoidable part of the development process. Even carefully written programs contain defects that cause unexpected behaviour, crashes, or incorrect results. Identifying a bug is only the first step. Understanding what kind of bug it is, why it happened, and how it should be fixed is equally important for improving software quality over time [5]. Different defect types call for different debugging strategies, resource allocations, and repair approaches [5], [13].

In practice, developers spend a significant portion of their time triaging and debugging reported issues. Bug categorization makes this process more efficient by grouping defects into meaningful categories that guide debugging strategies and resource allocation [2]. Early prediction of a bug’s category from its report helps teams prioritize work before examining the code [13]. Structured categorization also provides actionable measurements that managers and developers can use to improve their processes [6]. This process-level use depends on labels that are consistent and comparable across bugs, so that a collection of classified defects can be aggregated to reveal which kinds of mistake a development process keeps producing.

Manual triage still gives the best results, but it does not scale to large benchmark datasets [2]. Evaluating automated classification therefore requires a dataset that is large enough to support general conclusions and rich enough in evidence to classify from.

1.1 Defects4J: The Benchmark Dataset

Defects4J is a curated database of real, reproducible Java bugs paired with developer-written patches and regression test suites [17]. It contains over 830 active bugs across

17 open-source Java projects and serves as a standard benchmark for empirical software engineering research, including fault localization, automated program repair, and testing studies [23], [25].

Every bug in Defects4J comes with a rich set of artifacts: the original bug report, the triggering tests that expose the bug, the buggy and fixed source code, and the buggy-to-fixed diff. These artifacts provide several independent perspectives on the same defect. A stack trace shows where the program failed, a bug report describes what the user observed, a failing test states the expected behaviour, and a diff shows what the developer eventually changed. This richness makes Defects4J particularly well suited for automated classification research.

One property of Defects4J is central to the evaluation in this thesis. Because every bug ships with the developer’s actual fix, we can construct two versions of the same case file that differ in exactly one respect: whether the fix is visible. We refer to these as the *pre-fix* and *post-fix* versions of a bug, and define them precisely in Section 1.2. This controlled difference provides an evaluation reference in a dataset that carries no ODC labels, and Section 3.9 develops it into the evaluation framework used throughout this thesis.

1.2 Motivation

Despite the widespread use of Defects4J in software engineering research, no study has fully classified its bugs using a principled, semantic taxonomy. Existing classification efforts fall into three groups, each with a different focus and a different limitation.

First, structural approaches such as the work of Van der Spuy and Fischer [29] classify bugs by their control-flow and data-flow graph representations. This work covers 488 bugs, but it captures how bugs manifest syntactically rather than why they exist semantically. A missing null check and an omitted feature can produce similar control-flow changes while representing fundamentally different defect causes.

Second, patch-level taxonomies such as the Defects4J dissection by Sobreira et al. [27] classify repair actions for 395 bugs from an older version of the dataset. This describes what developers changed, not the defect mechanism that made the change necessary.

Third, small-sample inspections such as the work of Jiang et al. [16] manually analyse 50 bugs. The analysis is detailed, but the approach does not scale to the full benchmark.

None of these approaches classifies bugs into a semantic defect-type taxonomy at the scale of the full benchmark. This gap motivates an automated, scalable classification approach that goes beyond structural and patch-level patterns.

Large Language Models (LLMs) offer a promising way to address this gap. Recent work shows that code-aware LLM reasoning can classify software artifacts competitively, provided the prompt is designed well [7], [20], [24]. Left unconstrained, however, LLM classification suffers from label inconsistency: with no fixed label space, different runs assign incompatible categories to the same bug [20], and the resulting labels cannot be aggregated or compared. Supplying a taxonomy, and defining its labels explicitly, measurably changes what the model predicts [22]. Orthogonal Defect Classification (ODC) provides such a taxonomy, classifying each defect into one of seven Design/Code defect types that capture the semantics of the bug itself [6], [15]. Pairing LLMs with ODC turns classification into a bounded, semantically grounded decision problem.

A forced, fixed schema, however, introduces the opposite failure: it manufactures false certainty for defects that do not fit any of its categories. We therefore treat the coverage of the taxonomy as something to measure rather than assume, by giving the model an explicit escape category, “Other”. A defect that falls outside the seven types becomes recorded data instead of a mislabelled forced choice.

Pre-fix and post-fix classification. In this study, every bug is classified twice, from two versions of the same evidence that differ in exactly one respect.

- **Pre-fix classification** uses only the evidence available before the defect is repaired: the bug report, the failing tests, the exception and stack trace, and the relevant source-code context around the failure. This is the situation a developer faces during triage, before anyone has written a fix, and it is the classification the pipeline is intended to produce.
- **Post-fix classification** uses the same evidence together with the developer’s actual fix, in the form of the buggy-to-fixed code diff that Defects4J provides for every bug. Everything else in the evidence is identical.

Because an ODC defect type describes the nature of the correction made to fix a bug, the post-fix classification is the better-informed of the two. Since Defects4J carries no ground-truth ODC annotations, we therefore use the post-fix classification as a fix-informed evaluation reference: comparing the pre-fix and post-fix classifications of the same bug shows how well a label assigned before the fix agrees with a label

assigned once the fix is known. We compare the two labels at four tiers: whether they assign the same type (strict match), whether either type appears among the alternative types listed by the other classification (top-2 match), whether both types belong to the same broad group of related ODC types (family match), and Cohen’s κ , which corrects strict agreement for the agreement expected by chance. When the two labels differ for the same bug, we refer to this as *drift*. Section 3.4 describes how the two evidence modes are constructed, and Section 3.9 develops their comparison into the evaluation framework.

Classification conditions. Each classification is made under a *condition*, defined by two independent choices. The **taxonomy** determines the label space: *free*, in which the model describes the defect in its own words; *closed*, in which it must select one of the seven ODC types; or *open*, in which it may also select the “Other” escape category. The **strategy** determines how the model reasons: *zero*, a single plain prompt without a taxonomy or examples; *few*, a single prompt with type definitions, a decision checklist, and worked examples; or *scientific*, an enforced reasoning loop in which, on every turn, the model states a hypothesis about the defect type and a prediction, and then either requests a specific piece of additional evidence, called a *probe*, or concludes. A condition is named by its strategy and taxonomy, such as *scientific-open*, which is the default condition in this thesis. Section 3.5 defines the valid conditions, and Section 3.7 describes the reasoning loop.

1.3 Problem Statement

This section defines the specific research gap that this thesis addresses and formally states the problem we aim to solve.

1.3.1 Limitations of Existing Approaches

Current attempts at classifying Defects4J bugs suffer from five limitations:

1. **Coverage gap:** No study has fully labelled the active Defects4J set. Existing work covers only tens to a few hundred bugs [16], [27], [29].
2. **Structural-only labels:** Control-flow and data-flow analyses capture how bugs manifest syntactically, not the defect type they belong to [29].
3. **Patch-focused taxonomies:** Repair-action classification describes fix patterns, not the underlying defect mechanism [27].

4. **Post-fix dependency:** Machine learning approaches require labelled training data, which Defects4J does not have for ODC, or access to post-fix code, which limits their applicability to pre-triage scenarios [28].
5. **Evaluation scarcity:** Without ODC ground truth, there is no practical evaluation framework for large-scale automated classification [2].

1.3.2 The Research Gap

No study has produced a context-aware, semantically grounded, scalable classification of Defects4J. Addressing this gap requires a solution that is:

- **Context-aware:** using multi-modal evidence from code, tests, bug reports, and stack traces, rather than a single information source.
- **Semantic:** classifying defects by their defect mechanism, not by syntactic patterns.
- **Taxonomy-constrained:** using a principled framework such as ODC to prevent label inconsistency and to make results comparable across studies.
- **Open to its own limits:** able to record a defect that does not fit, rather than forcing it into the nearest category.
- **Scalable:** using LLM-based reasoning to classify at benchmark scale without manual labelling.
- **Evaluable without ground truth:** providing its own evaluation reference, since Defects4J supplies none.

1.3.3 Problem Definition

Given these limitations, we define the problem as follows: *how can we classify Defects4J bugs into ODC Design/Code defect types from pre-fix evidence alone, while also being able to evaluate that classification without ground-truth labels?*

The four research questions of Section 1.5 decompose this problem into measurable parts. Our guiding intuition is that pre-fix classification is a prediction made from symptoms alone, while post-fix classification is a reference made with the fix visible; agreement between the two indicates that the prediction is stable enough to justify operational use of the pre-fix signal.

1.4 Research Objectives

This thesis addresses the following four research objectives:

- RO-1:** Build an automated pipeline that classifies Defects4J bugs into ODC defect types using LLM reasoning grounded in evidence the model must actively request.
- RO-2:** Collect bug evidence in two modes, pre-fix (symptoms only) and post-fix (with the actual code fix), mapping Defects4J artifacts onto ODC’s opener and closer attributes, which are the attributes recorded when a defect is found and when it is fixed.
- RO-3:** Separate the label space the model uses from the way it reasons toward a label, producing a small set of controlled, comparable classification conditions.
- RO-4:** Design a four-tier evaluation framework that compares pre-fix and post-fix classifications using strict match, top-2 match, family match, and Cohen’s κ .

1.5 Research Questions

We evaluate these objectives through four research questions.

- RQ1: Defect-type distribution across projects.** What is the distribution of ODC defect types across the Defects4J benchmark, and does this distribution vary significantly across projects?
- RQ2: Coverage of the ODC taxonomy.** Do the seven standard ODC defect types cover all the bugs in Defects4J, or do some bugs fall outside them and require the “Other” escape category?
- RQ3: Classification accuracy.** Treating the post-fix classification as a fix-informed reference, how often does the pre-fix classification assign the same defect type, and when it does not, how far apart are the two labels under a four-tier evaluation?
- RQ4: Contribution of each component.** **RQ4a:** What does explicit ODC taxonomy grounding contribute, compared with letting the model describe the bug in its own words? **RQ4b:** What does the enforced hypothesis–prediction–probe–observation loop contribute over a strong single-call few-shot prompt, when both use the same taxonomy?

RQ1 and RQ2 characterize the dataset and the fit of the taxonomy to it; RQ3 measures how closely the default configuration reproduces the fix-informed reference and how

the labels change when the fix is visible; and RQ4 isolates what each structured component contributes.

1.6 Contributions

The main contributions of this thesis are:

- We separate the label space the model uses (the taxonomy) from how it reasons toward a label (the strategy), giving a small set of controlled, comparable conditions instead of one all-in-one prompt.
- We give ODC an open “Other” escape category that requires a written justification, which turns the coverage of the taxonomy into something we measure directly instead of assume.
- We build an enforced scientific reasoning loop, adapted from fault localization to defect-type classification, in which the model has to commit to a hypothesis and probe held-back evidence before it is allowed to conclude, leaving a full, auditable reasoning transcript.
- We classify every bug in two evidence modes, pre-fix and post-fix, mirroring ODC’s opener/closer split, which lets us evaluate the pipeline without ground-truth labels.
- We report a quantitative evaluation over 1,640 classifications of 410 bugs from six projects, with confidence intervals, permutation tests, and paired bootstrap comparisons.
- We study the enforced loop against a strong few-shot prompt bug by bug, not only in aggregate: we count the bugs the loop improves and degrades, relate the gain to the held-back evidence the loop requests, and read the reasoning transcripts of both strategies under two LLMs to show where the loop helps, where it falls short, and which of these behaviours come from the pipeline rather than from the model.

1.7 Report Organization

The research gap and the problem statement are given in Section 1.3 above. The rest of this report is organized as follows. Chapter 2 reviews the literature on Defects4J classification, the ODC framework, LLM-based bug categorization, and human agreement

on ODC labels. Chapter 3 describes the proposed methodology. Chapter 4 explains the experimental design, the data, and the statistical methods. Chapter 5 presents the results for all four research questions. Chapter 6 discusses threats to validity. Finally, Chapter 7 concludes the report with a summary, limitations, and future work.

Chapter 2

Related Works

This chapter reviews the existing literature that forms the foundation of our research. We organize the review around five themes: existing attempts at classifying Defects4J bugs, LLM-based defect classification, the need for a principled taxonomy, the Orthogonal Defect Classification framework that our pipeline adopts, and human agreement on ODC labels.

Defects4J [17] is one of the most widely used benchmark datasets for software debugging and defect analysis research. It provides a curated collection of reproducible real-world bugs from 17 open-source Java projects, along with their original bug reports, reproducible failures, buggy and fixed source code, and buggy-to-fixed patches. This rich and reproducible set of artifacts makes Defects4J particularly suitable for automated defect classification. In this research, we leverage these artifacts to investigate the classification of Defects4J bugs into ODC defect categories using LLMs.

2.1 Existing Defects4J Bug Type Classification

Several studies have attempted to classify bugs in the Defects4J dataset, each using a different approach and covering a different subset of the data.

Van der Spuy and Fischer [29] manually classified 488 faults using control-flow and data-flow graph abstractions, identifying six control-flow and two data-flow categories. This structural perspective provides insight into how defects alter program execution; however, it does not necessarily capture the underlying reason for the defect. For example, a missing null check and an omitted feature may result in similar control-flow changes while representing fundamentally different defect causes.

Sobreira et al. [27] analysed 395 bugs from Defects4J v1.2, focusing on repair actions and patch patterns. Their findings showed that a substantial proportion of patches involve modifications to conditional blocks. Although patch-based analysis is valuable for understanding how developers repair defects, patch characteristics primarily describe the changes made to the program rather than the underlying defect mechanism that necessitated those changes.

Jiang et al. [16] manually examined 50 Defects4J bugs to investigate test-suite quality and fault fixability. While their analysis provides detailed insights into individual defects, the relatively small sample limits the generalizability of the findings to the broader Defects4J dataset.

These studies provide valuable perspectives on defect structure, repair actions, and fixability. However, they examine relatively limited subsets of Defects4J, rely primarily on code-structural or patch-based taxonomies, and do not apply a semantic defect-type taxonomy to the full set of bugs. Our work addresses this gap by investigating ODC-based semantic defect classification across the Defects4J dataset using LLMs.

2.2 LLM-based Defect Classification

Recent studies have demonstrated that LLMs can effectively classify software defects when provided with relevant contextual information.

Colavito et al. [7] showed that LLMs can classify issue reports, which are closely related to bug reports, with reasonable accuracy, validating the basic hypothesis that LLMs are suitable for defect classification tasks. Koyuncu [20] extended this by using LLMs and several prompt engineering techniques, including few-shot, chain-of-thought, tree-of-thoughts, and plan-and-solve prompting, to achieve fine-grained bug report categorization.

Li et al. [22] and Du et al. [8] use LLMs, leveraging several prompting strategies, to classify bug reports into categories based on their behaviour and reproducibility. These studies showed that LLMs benefit from structured prompting strategies, ranging from simple task prompts to reasoning-based approaches such as chain-of-thought and tree-of-thought, enriched with few-shot examples and domain knowledge integration, to improve the accuracy, generalization, and interpretability of bug report categorization. Nong et al. [24] similarly showed that chain-of-thought prompting substantially improves LLM performance on vulnerability analysis.

Relatedly, Kang et al. [19] incorporated scientific debugging into LLM prompting

through AutoSD. In their approach, the model formulates a hypothesis, performs a debugging experiment to test it, and uses the resulting evidence to determine whether the hypothesis is supported, primarily for explainable fault localization. Kang et al. [18] later extended this direction to a tool-using setting, in which the LLM gathers code and test evidence on demand through function calls rather than receiving a fixed context. Building on these ideas, our iterative reasoning process uses targeted probes of the collected evidence to evaluate competing hypotheses and ultimately classify the defect according to ODC type. The difference lies in the task: these works use evidence to decide *where* a fault is, whereas we use it to decide *what kind* of defect it is.

Compared with traditional machine learning approaches, LLMs offer a key advantage in their ability to jointly interpret natural-language and source-code information. Earlier automated ODC classification systems based on SVM and Naive Bayes achieved accuracies of approximately 77–82% and were limited by sparse textual representations [14], [28]. In contrast, LLMs can integrate multiple forms of software engineering evidence, including code structure, test behaviour, and error messages, within a single inference process. Moreover, their ability to perform classification without task-specific labelled training data makes them particularly suitable for applying semantic defect taxonomies such as ODC to large-scale bug datasets.

Existing defect classification approaches primarily rely on bug reports as the source of evidence for classification [7], [8], [20], [22]. In contrast, our approach integrates multiple software artifacts, including bug reports, source code, failing tests, exceptions, and stack traces, to classify defect types. The use of diverse evidence motivates the need for a well-established and systematically defined taxonomy to constrain and guide LLM-based classification. We therefore adopt ODC as the underlying taxonomy for organizing and interpreting defect types.

2.3 The Taxonomy Problem

While LLMs show promise for defect classification, using them without a structured taxonomy leads to unreliable results. Without standardization, different LLM runs produce different category labels for the same bug [20]. Machine-learning-based bug classification studies also suffer from a proliferation of incompatible taxonomies, which makes it difficult to compare results across studies [2]. Classification without a principled framework produces vague labels with limited operational value [28].

Direct evidence that a fixed taxonomy helps comes from Li et al. [22]. Their KnowBug

framework predicts bug types from deep-learning-framework bug reports, and they compare a prompt that states only the task with one that adds explicit definitions of every label in the taxonomy. Adding the label definitions raises the F-measure from 26.44% to 33.17% on TensorFlow, from 29.08% to 41.39% on MXNET, and from 17.81% to 31.14% on PaddlePaddle. They also found that prompts with an explicit reasoning structure outperformed prompts without one on all three projects.

These findings motivate the use of a well-established taxonomy to constrain and guide LLM outputs. In this thesis, that taxonomy is ODC, and Section 5.5.1 measures the effect of removing it on our own data.

2.4 Orthogonal Defect Classification (ODC)

ODC was introduced by Chillarege et al. [6] as a framework that classifies defects along orthogonal (independent) attributes. It was originally designed to give an in-process signal during active development. The ODC framework defines eight attributes organized into two phases: *opener* attributes recorded at discovery time and *closer* attributes recorded at fix time, summarized in Table 2.1.

Table 2.1: ODC opener and closer attributes [15]

Phase	Attribute	Description
Opener	Activity	The specific activity being performed when the defect was found, such as code inspection or unit test
	Trigger	The condition that exposed the defect
	Impact	The user-visible consequence of the defect
Closer	Defect Type	The semantic nature of the actual correction made
	Target	Design or Code
	Qualifier	Missing, Incorrect, or Extraneous
	Age	Whether the faulty code is base, new, rewritten, or refixed
	Source	Whether the faulty code is developed in-house, reused from a library, outsourced, or ported

The central attribute for our work is the **Defect Type**, which represents the semantic nature of the code correction made to fix a bug. For Design/Code targets, ODC defines seven defect types, which we group into two families. Table 2.2 lists them with their definitions.

Table 2.2: The seven ODC Design/Code defect types [6], [15], grouped by this study into two families

Defect Type	Description
<i>Family: Control and Data Flow</i>	
Algorithm/Method	The logic, sequence, or efficiency of a computation is wrong. The fix rewrites how a result is produced, without needing a design change
Assignment/Initialization	A value is assigned, initialized, or declared incorrectly, or not at all. The fix corrects a small number of lines of data handling
Checking	A validation, guard, boundary test, or error condition is missing or wrong. The fix adds or corrects a conditional
Timing/Serialization	Resources are managed incorrectly under concurrency: missing or wrong serialization, locking, or timing of shared-resource access
<i>Family: Structural</i>	
Function/Class/Object	Capability is missing or substantially wrong at the design level. The fix requires a formal design change affecting significant capability
Interface/O-O Messages	The defect lies in the interaction between components: parameters, method contracts, or message passing across a module boundary
Relationship	An association between objects, data structures, or entities is wrong, such as a conditional dependency between two entities that was not honoured

Two of these types are central to the interpretation of our results. **Checking** covers a defect in a validation or guard: the program failed to test a condition it should have tested, or tested it incorrectly. **Algorithm/Method** covers a defect in the computation itself: the program tested the right conditions but computed the wrong result. Many real fixes can be described either way. Adding a bounds check to a loop can be read as supplying a missing guard, or as correcting a computation that should never have gone out of bounds. Although ODC intends its categories to be mutually exclusive, this boundary is difficult in practice. Section 5.4.3 shows that it accounts for the majority of the label changes we observe, and Section 2.5 shows that human raters find related boundaries equally difficult.

ODC defect types remain actively used in modern software engineering research. Es-

fahani et al. [10] applied ODC to classify defects in LLM-generated code in 2024, demonstrating that the taxonomy generalizes beyond human-written software. The official ODC v5.2 specification from IBM [15] continues to serve as the reference standard for the framework.

2.4.1 Impact, and Why This Study Does Not Classify It

Impact records the consequence the defect would have for the customer. For an in-process defect, the rater selects the impact the defect would have had if it had escaped to the field; for a defect reported from the field, the impact it actually had [15]. The v5.2 specification defines thirteen values: Installability, Integrity/Security, Performance, Maintenance, Serviceability, Migration, Documentation, Usability, Standards, Reliability, Requirements, Accessibility, and Capability. Two of these cover most ordinary software defects. **Reliability** covers a failure to perform the intended function without unplanned interruption, and the specification states that severe interruptions such as abnormal termination are always Reliability. **Capability** covers a defect where the software fails to perform its intended function and satisfy known requirements, but the customer is not affected in any of the other ways.

Impact and Defect Type sit on opposite sides of ODC’s two-phase model and describe different aspects of the same defect. Impact is an opener attribute that describes the consequence to the user, while Defect Type is a closer attribute that describes the nature of the correction. A single defect type can produce many impacts, and a single impact can arise from any defect type: a missing bounds check (Checking) might surface as a crash (Reliability) in one context and as a wrong result (Capability) in another.

This study classifies Defect Type only, and no Impact results are reported in this thesis. Impact is judged primarily from the bug report, which describes what a user observed, and Defects4J does not supply bug reports uniformly: availability varies by project and by issue tracker, and is notably lower for projects whose reports are hosted on archived trackers. Assigning an Impact value from a failing unit test alone would mean inferring a customer consequence from evidence that does not contain one. Defect Type, by contrast, can be inferred from the evidence that Defects4J provides for every bug: the code, the failing test, and the fix. Impact classification is therefore left to future work (Section 7.3).

2.5 Human Agreement on ODC Classification

An evaluation of an automated ODC classifier requires some expectation of how consistently the task can be performed. Several secondary sources state that human ODC experts agree with each other roughly 70–80% of the time.

The available literature instead indicates that agreement on ODC-style classification depends strongly on how much evidence the rater has. Table 2.3 summarizes the relevant studies, ordered by the evidence available to the raters.

Table 2.3: Reported human agreement on ODC-style defect classification, ordered by the evidence available to the raters

Study	Evidence available	Agreement reported
Henningsson and Wohlin [11], 8 raters, 30 faults	Written fault descriptions only	Mean pairwise $\kappa = 0.16$; merging the two most-confused classes reached only 0.24
Hernández-González et al. [12], 5 annotators	Defect report text only	A simple majority was reached on only 481 of 962 and 394 of 675 defects
El Emam and Wiczorek [9]	Code available at detection time	The scheme is “in general repeatable”
Rahman and Farhana [26], 2 coders	Full repository context	72.7% agreement, $\kappa = 0.70$ (open coding); 95.1%, $\kappa = 0.93$ (closed coding)
Agnelo et al. [1]	Full code and change context	$\kappa = 0.93$ for Defect Type

Raters working from written descriptions alone reach $\kappa = 0.16$, while raters with access to the code and the change reach $\kappa = 0.93$. On the Landis–Koch scale [21], this is a range from slight to almost perfect agreement, determined largely by the evidence available.

This finding is relevant to our study in three ways. First, Henningsson and Wohlin [11] attribute the low agreement in their study to the limited fault descriptions and recommend providing raters with source code and change information, which corresponds to our post-fix evidence mode. Second, if agreement increases with evidence, the classification that sees the developer’s fix is the better-informed one, which supports its use as the reference in RQ3. Third, the most frequent confusion reported by Henningsson and Wohlin [11] was between Assignment and Algorithm, with Checking and Algorithm third, and Section 5.4.3 shows that our own label changes fall within the same group of types.

The confusion counts must be read carefully. In Henningsson and Wohlin [11], the most-confused pair was Assignment and Algorithm, and the reported figure of 22 out of 28 refers to rater pairs, not to bugs or classifications. By occasions, the ranking was Assignment–Algorithm 184, Interface–Algorithm 77, Checking–Algorithm 68, Assignment–Interface 59, and Assignment–Checking 49.

2.6 Summary

Despite the growing adoption of LLMs in software engineering, their application to semantic defect classification using the ODC taxonomy remains largely unexplored. Existing defect classification approaches have primarily relied on individual sources of evidence, such as issue reports, code-level features, or repair patches, rather than integrating diverse software artifacts, including bug reports, source code, failing tests, exceptions, and stack traces. Furthermore, there is limited understanding of how effectively LLMs can classify defects into well-defined ODC categories and how different prompting strategies influence classification performance. The lack of ODC defect-type annotations in the widely used Defects4J benchmark also presents a significant challenge for systematic evaluation. These gaps motivate our study to investigate LLM-based ODC defect classification using multiple software artifacts and to systematically compare different prompting strategies for improving the accuracy and reliability of defect-type classification.

Chapter 3

Proposed Methodology

This chapter describes the design and implementation of our classification pipeline. The Defects4J v2.0 dataset [17] provides over 830 reproducible bugs curated across 17 real-world Java projects. In this research, we classify Defects4J bugs into ODC defect types using an LLM, following three stages: Data Collection, Bug Classification, and Evaluation. The primary goal of this research is to identify and categorize software defects based on their underlying characteristics, thereby providing a structured understanding of the nature and distribution of defects in the Defects4J dataset. The data collection stage prepares and transforms the Defects4J bug data into a suitable representation for classification; the classification stage assigns appropriate ODC types to the identified bugs; and the evaluation stage assesses the performance and reliability of the classification results using appropriate evaluation measures. Figure 3.1 demonstrates the workflow, and the rest of this chapter describes each stage in turn.

3.1 Pipeline Architecture Overview

The pipeline follows a file-oriented, synchronous architecture with no database, queue, or asynchronous orchestration. Each stage reads and writes structured JSON artifacts, so any stage can be rerun, inspected, or audited independently.

1. **Data Collection:** We build a case file for each bug containing the report, the failing tests, the error messages, and the surrounding code, which is the information a developer would examine before seeing the fix. The output is a structured `context.json` file per bug.
2. **Classification:** The case file is provided to an LLM, which assigns an ODC defect type using one of three prompting strategies, ranging from a single

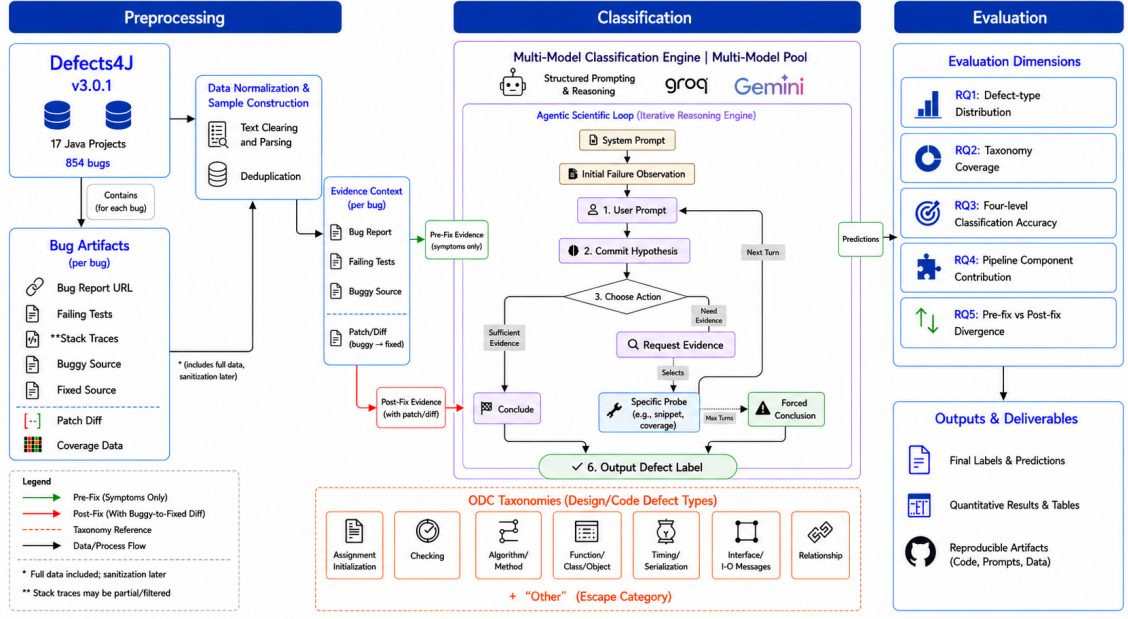


Figure 3.1: Overview of the proposed methodology: data collection from Defects4J artifacts into per-bug evidence contexts, ODC classification through the enforced reasoning loop, and evaluation across the research questions

prompt to an enforced, evidence-driven reasoning loop. The output is a `classification.json` file per bug per condition.

3. **Evaluation:** Because Defects4J carries no ODC labels, we classify each bug a second time with the real fix visible, and measure how closely the pre-fix classification matches the fix-informed one.

Two architectural principles hold throughout. First, the pipeline supports multiple LLM providers (Gemini, OpenRouter, Groq, and OpenAI-compatible APIs), so the classification logic is independent of any one vendor. Second, the list of classes modified by the fix is stored as a hidden oracle and never enters the LLM prompt, which prevents the pre-fix classification from accessing fix information [25].

3.2 ODC and Defects4J Artifact Mapping

The pipeline maps Defects4J artifacts to ODC attributes following the framework’s two-phase opener/closer model, as shown in Figure 3.2.

On the opener side (discovery story):

- **Tests** map to **Activity**, because the test type indicates what was being performed when the defect was found. For Defects4J, this is a unit test in every case.

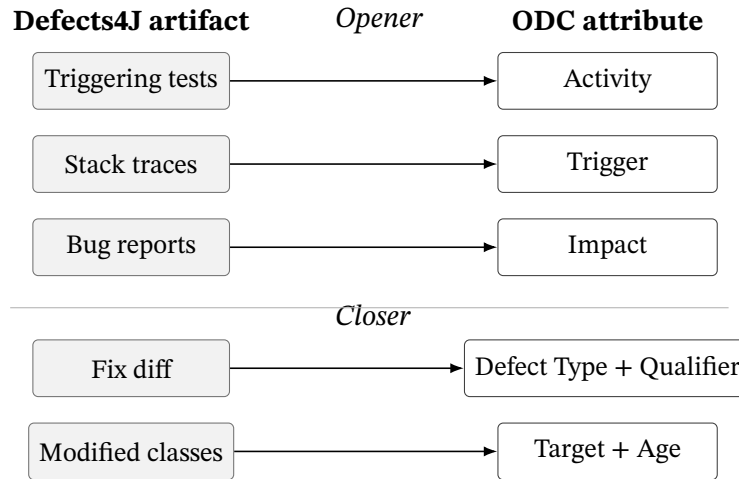


Figure 3.2: Mapping between Defects4J artifacts and ODC opener/closer attributes. Impact is shown for completeness; this study classifies Defect Type only (Section 2.4.1).

- **Stack traces** map to **Trigger**, because the runtime context captured in the trace shows the specific condition that exposed the defect.
- **Bug reports** map to **Impact**, because they describe the user-visible symptom and consequence, such as a crash or an incorrect result.

On the closer side (repair story):

- **Code diffs** map to **Defect Type** and **Qualifier**, because the diff reveals the semantic nature of the mistake and how it manifested (Missing, Incorrect, or Extraneous).
- **Modified classes** map to **Target** and **Age**, because they show where the fix was applied and, with commit history, when the defect was likely introduced.

Two attributes are fixed by the construction of the dataset rather than classified. Target is always Code, because Defects4J bugs are fixed in source and there is no separate design document. Source would be Developed In-House for nearly every project, since these are long-running, community-maintained Java libraries.

This separation follows ODC’s design principle that discovery data should not be mixed with repair data [6]. It also underlies our evaluation: the pre-fix classification sees opener-side evidence, the post-fix classification additionally sees closer-side evidence, and the Defect Type is a closer attribute.

3.3 Evidence Collection

This section describes how the evidence for the pre-fix and post-fix classifications defined in Section 1.2 is collected. To classify a Defects4J bug into an ODC type using an LLM, we prepare two types of files, namely a pre-fix context and a post-fix context, for the buggy and fixed versions of the code respectively. The only difference between these two files is the fix diff, which represents the exact difference between the buggy version of the code and its fixed version. The pre-fix context is the one used for bug classification, without access to any information from the corresponding fix; the post-fix context is used to produce the evaluation reference, where the actual code changes provide sufficient evidence for defect-type classification.

For each bug, we extract and store the following artifacts as the context for the LLM.

Bug identity and Defects4J metadata. We record the project and bug IDs, the buggy and fixed revision identifiers, the issue-tracker report ID and URL, the modified classes, and the triggering and relevant tests.

Bug report content. We fetch the original bug report from the issue tracker (JIRA, GitHub, or a generic web page) and store it in full, including its title, labels, and comment thread.

Failing tests, exception, and stack trace. We check out the buggy revision, compile it, and run the test suite to confirm that the bug reproduces. For each triggering test, we capture its name, the raised exception, the raw stack trace, and a parsed list of every frame with its class, method, file, and line.

Suspicious frames. A single failing JUnit test typically produces a long stack trace, most of whose frames belong to the test harness (JUnit, TestNG, Hamcrest), the JDK, or the build tool (Ant, Maven). These frames carry no information about the defect. A suspicious frame is a stack frame that points into the project's own source tree. We identify suspicious frames by filtering the parsed frame list against the project's source roots, preserving the original stack order so that the innermost project frame comes first. Suspicious frames serve two purposes: they are the anchor points for snippet extraction, and therefore determine which code the model reads; and their order encodes the call path that led to the failure.

Code snippets. Around each suspicious frame and each failing test, we extract a short source window with the relevant line marked. Production snippets cover ± 12 lines around the frame, and test snippets cover ± 18 lines, because a test assertion is usually further from its setup. These snippets provide localized context showing expected versus actual behaviour.

Coverage (optional). Cobertura coverage analysis can be run on the suspicious classes, and the resulting XML is parsed for line and branch execution rates. Coverage is available to the model as a probe target rather than included by default.

Normalization and deduplication. Bug report HTML is converted to text, tracker boilerplate is removed, and whitespace is collapsed, so that the same report does not consume a different amount of the prompt depending on its tracker. Repeated frames from recursive calls are collapsed, and a stack-trace prefix shared by several triggering tests is stored once. Code snippets are deduplicated by source range, so two suspicious frames in the same method do not produce overlapping copies. As a result, each distinct piece of evidence appears once in the case file.

Hidden oracles. We separately store the modified classes as a hidden oracle. This field is excluded from the LLM prompt in both evidence modes and is used only during evaluation, for example to check whether a probe reached the modified class.

Fix diff (post-fix oracle). For post-fix classification, we also record the buggy-to-fixed source diff. This field is empty in the pre-fix record and withheld from the model otherwise.

Keeping the fix out of the pre-fix evidence. Two channels can leak fix information into the pre-fix context. The raw Defects4J metadata names the classes the fix modified, and bug reports for resolved issues often discuss the resolution, including status transitions and “fixed in” comments. Both are removed from the pre-fix evidence before it is shown to the model, while the post-fix evidence is left unchanged. The two case files therefore differ only in the fix information, which is the difference the pre-fix/post-fix comparison is designed to isolate.

3.4 Evidence Modes: Pre-fix versus Post-fix

The pre-fix and post-fix classifications defined in Section 1.2 correspond to the two phases of ODC’s opener/closer model [6], as shown by the artifact mapping in Figure 3.2.

Pre-fix mode (the default) corresponds to the opener side. The evidence it uses (stack traces, test failures, error messages, code snippets, and bug report content) describes how the defect was discovered and what went wrong, so the LLM must reason about the defect from its symptoms. Conceptually, this is like a doctor diagnosing a patient from symptoms alone.

Post-fix mode corresponds to the closer side. The buggy-to-fixed diff shows exactly what was changed, which is the evidence ODC uses to record the Defect Type. This is like a pathologist with biopsy results who can see the exact problem.

Because the Defect Type is a closer attribute, the post-fix mode holds the evidence that defines it, while the pre-fix mode must infer it. If the pipeline classifies correctly from symptoms alone, the pre-fix result should therefore broadly agree with the post-fix result.

3.5 LLM-based ODC Classification

In the classification stage, Defects4J bugs are classified into ODC types using an LLM. A single prompt combines two separate decisions: the label space the model may answer in, and the way it is asked to reason toward an answer. We separate these into two independent axes, the taxonomy axis and the strategy axis.

3.5.1 Taxonomy Axis: How Strict the Vocabulary Is

The model can answer in three ways:

- **Free:** The model answers in its own words with no fixed list. This setting allows us to measure how much a taxonomy narrows an otherwise unconstrained vocabulary.
- **Closed:** The model must select exactly one of the seven canonical ODC Design/Code types.
- **Open:** The model may select one of the seven types or “Other”. An “Other” answer is accepted only together with a written justification, the nearest of the

seven types, and a confidence score, so that every escape is auditable.

Whenever a fixed list is used, the model also returns a structured record: the chosen type, close alternatives, the ODC family, a confidence score, and a short reasoning trace. Open is the default taxonomy setting, because we do not force a bug into a fixed schema; a defect that does not belong to any ODC type is recorded as data.

3.5.2 Strategy Axis: How the Model Reasons

Zero: a single plain prompt. The simplest strategy is one plain prompt with no taxonomy and no examples. It is the unstructured baseline against which the other strategies are measured, and it is taxonomy-free by definition.

Few: one strong, well-prepared prompt. The second strategy also asks for a single answer, but the prompt is built carefully. It defines every ODC type with example patterns, instructs the model not to default to the general Function/Class/Object type, walks through a short decision checklist before that type may be considered, and includes worked examples chosen for pairs of types that are easy to confuse. This is the strongest single-call prompt we could build, and it is the baseline the third strategy has to improve upon.

Scientific: an enforced reasoning loop. The third strategy does not allow the model to simply narrate a chain of thought. Instead, the model must commit to a hypothesis, request specific evidence, and evaluate what it receives before concluding. Section 3.7 describes the loop in detail.

3.5.3 Valid Conditions

Not every combination of the two axes is meaningful. A zero-shot prompt carries no vocabulary constraint, and the free setting defines none, so only five of the nine combinations are valid. Figure 3.3 shows the condition space.

We selected *scientific-open* as the default for two reasons. First, forcing every bug into a fixed seven-type list manufactures false certainty for bugs that do not fit, whereas the open vocabulary, with its mandatory justification for every escape, turns taxonomy coverage into something we measure. Second, in early testing, simply narrating a step-by-step reasoning style left the model’s answers unchanged; only when that reasoning was enforced, by requiring a committed prediction before new evidence was revealed,

	free	closed	open
zero	zero-free	–	–
few	–	few-closed	few-open
scientific	–	scientific-closed	scientific-open

Figure 3.3: The three classification strategies (rows) crossed with the three vocabulary settings (columns). Five of the nine cells are valid; the four grey cells are invalid combinations, since *zero* carries no vocabulary constraint and *free* defines none. *scientific-open* (bold) is the default for both pre-fix and post-fix evidence.

did a meaningful share of labels change, and those changes moved toward the fix-informed reference.

3.6 Prompt Design

Every classification combines a **system prompt**, which holds the instructions that are constant across bugs within a condition, with a **user prompt**, which holds the evidence for one bug. Because the full prompts are long, we present them as templates that separate fixed instructions from dynamically filled components [3]. Figures 3.4 and 3.5 show the templates of the two taxonomy-grounded strategies; the *zero* strategy sends the same evidence with a request to describe the defect in the model’s own words and contains no ODC terms.

The highlighted components are defined as follows. **BUG_CONTEXT** is the evidence payload of Section 3.3 (sanitized bug report, failing tests with filtered stack traces, suspicious frames, and code snippets, plus the fix diff in post-fix mode), identical for both strategies. **ODC_TAXONOMY** gives each of the seven types with its family, definition, indicators, contrastive “when not to choose” guidance, and examples, and adds “Other” as a justified last resort under the open taxonomy. **ANTI_BIAS_RULES** instruct the model not to default to Function/Class/Object and not to rely on benchmark familiarity, without naming any expected distribution. **DECISION_TREE** asks seven diagnostic questions, one per type, with Function/Class/Object last, and **FEW_SHOT_EXAMPLES** are five contrastive worked examples [20]; both appear only in the *few* strategy. **PROBE_CATALOGUE** and **TURN_SCHEMA** define the evidence requests and per-turn structure of the *scientific* strategy (Section 3.7). **OUTPUT_SCHEMA** is the classification record: type,

family, confidence, alternative types with reasons, reasoning summary, evidence used and missing, a human-review flag that the model sets when it judges the evidence insufficient, and, under the open taxonomy, the escape justification.

```
[SYSTEM] You are an expert software defect analyst specializing in ODC.
Classify one bug into exactly one ODC type using ONLY the provided evidence.
DIFF_GUIDANCE          # post-fix only: how a change maps to a type
ANTI_BIAS_RULES  ODC_TAXONOMY  Return JSON matching OUTPUT_SCHEMA.
Work through these questions in order: DECISION_TREE
FEW_SHOT_EXAMPLES          # symptom, snippet, label, "NOT <type>"
[USER]   Classify this bug. Evidence mode: EVIDENCE_MODE (pre-fix | post-fix)
Rules: use ONLY this evidence; follow the decision process; flag incomplete
evidence for review; "Other" only as a justified last resort.
Evidence: BUG_CONTEXT
[MODEL]   OUTPUT_SCHEMA          # single call
```

Figure 3.4: Prompt template of the *few* strategy: all guidance and evidence in a single call. Highlighted names are dynamically filled components; text after # is explanatory.

```
[SYSTEM] You are an expert ODC defect analyst working as a scientific-debugging
agent. The failure is already observed (first user message). Each turn: state a
HYPOTHESIS and a PREDICTION, then EITHER request one probe OR conclude.
Probes: PROBE_CATALOGUE # list_evidence, stack trace, snippet, coverage, report
Commit the prediction BEFORE seeing the result; revise if refuted.
ANTI_BIAS_RULES  ODC_TAXONOMY  Respond with TURN_SCHEMA.
[USER]   Initial failure observation: BUG_CONTEXT
loop t = 1..6:
  [MODEL] { hypothesis_t, prediction_t, action_t, probe_t }
  [USER]   Observation: PROBE_RESULT_t # from the harness; must conclude by t = 6
until action_t = conclude -> OUTPUT_SCHEMA
```

Figure 3.5: Prompt template of the *scientific* strategy: the same bug context, but the decision tree and examples are replaced by a turn protocol in which further evidence is revealed only through probes the model requests.

3.7 Scientific Debugging Protocol

The scientific debugging protocol is the core reasoning strategy of the *scientific* condition. This section describes its origin and the enforced reasoning loop that implements it.

Our protocol is based on AutoSD [19], which applies scientific debugging to LLM prompting for explainable fault localization. In AutoSD, the initial failure observation, consisting of the failing test, error message, and code, is provided as the prompt input, and each subsequent iteration runs a cycle of hypothesis, prediction, experiment, observation, and conclusion.

We adopt the AutoSD loop-iteration structure and adapt it from localizing a fault to classifying its type. In our pipeline, the initial failure observation is a summary of the evidence context: the bug report, the failing tests with shortened stack-trace excerpts, the suspicious frames, and a limited number of code snippets and coverage entries.

The remaining collected evidence, such as complete stack traces, further source snippets of a class, and full coverage data, is held back. We refer to the full collected evidence for a bug as the *evidence store*; the model can access its held-back part only by issuing probes. On every turn the model must:

1. **Hypothesis:** Commit in writing to a specific claim about the defect type, before seeing any new evidence.
2. **Prediction:** State what specific evidence would have to be present if the hypothesis were correct.
3. **Experiment:** Request that evidence by issuing one of the probes in Table 3.1.
4. **Observation:** Read what the probe returned. The returned payload is recorded in the transcript, truncated to 2,000 characters.
5. **Conclusion:** Judge whether the observation supports or contradicts the hypothesis, and either conclude or begin another turn, up to a maximum of six turns.

The model can issue five probes, listed in Table 3.1. Each probe is executed by the pipeline against the evidence store and returns the recorded evidence; the model cannot see anything beyond what the initial summary and its probes return. If a probe's argument matches nothing, the probe returns the list of available items instead, so the model can correct the request on its next turn, and a probe that repeats an earlier request is not served again.

Table 3.1: The evidence probes available to the scientific strategy

Probe	Argument	Returns
list_evidence	none	An inventory of the available evidence: failing test names, classes with production and test snippets, classes with coverage data, and whether a bug report exists
full_stack_trace	test name	The complete stack trace and failure message of the matching failing test
snippet	class name	All source snippets collected for the matching class, including test sources
coverage	class name	The full line and branch coverage data of the matching class
bug_report	none	The full bug report text, sanitized of fix-related content in pre-fix mode

Figure 3.6 shows the cycle. The initial failure observation enters once, as the prompt

input; the dashed edge is taken when the evidence gathered so far is not sufficient to conclude.

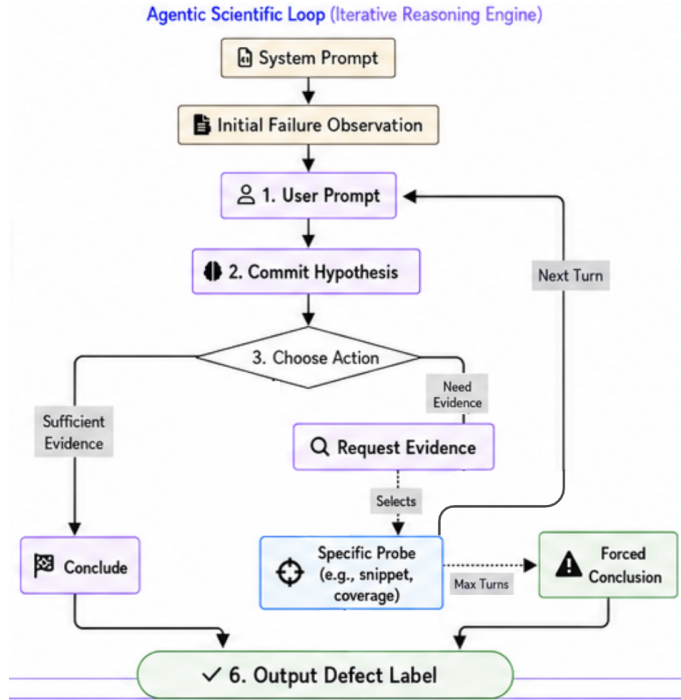


Figure 3.6: The enforced reasoning loop, adapted from the AutoSD loop-iteration structure [19]. The failure observation is the prompt input; each turn then runs hypothesis, prediction, experiment, observation, and conclusion.

Because the model must commit to a prediction before it is allowed to see the evidence that would confirm or refute it, this strategy cannot simply rationalise a first guess after the fact. This distinguishes it from a narrated chain of thought, in which the reasoning and the conclusion are produced together and the reasoning can be written to fit the conclusion.

If the loop reaches its turn limit without concluding, the model is required to conclude from the evidence it has, and this is recorded as a forced conclusion. Every turn, probe, returned payload, and hypothesis revision is written into the classification artifact, so the reasoning chain can be audited. Section 5.5.2 uses these transcripts to measure how often the loop reached the modified code.

3.7.1 An Example Turn

The following abstracted example illustrates the mechanism. Consider a bug whose visible symptom is an assertion failure comparing two numeric values, with a stack trace pointing into a utility method.

Hypothesis. “The defect is *Checking*: the method does not guard against an empty input range, so it proceeds into a computation that is undefined for that input.”

Prediction. “If this is correct, the method body will contain an arithmetic expression over the range with no preceding emptiness or bounds test.”

Experiment. The model issues a probe for the source of the named method, which it has not yet been shown.

Observation. The returned snippet contains a bounds test, but the arithmetic that follows uses the wrong comparison operator.

Conclusion. The prediction is refuted, because the guard exists. The model revises its hypothesis to *Algorithm/Method* and, on the next turn, probes the failing test to confirm that the expected value is consistent with a corrected operator.

The hypothesis is written down in a specific, falsifiable form before the evidence that refutes it is requested. The observation therefore comes from the evidence store after the prediction is fixed, rather than being generated together with the conclusion. Section 5.5.2 measures the effect of this design.

3.8 Classification Pipeline and Validation

Whatever the model returns is checked before it is used. We confirm that the type is one of the seven canonical names or a properly justified “Other”; clamp the confidence score into a valid range; recompute the ODC family from the chosen type rather than trusting the model’s own family label; and normalize the alternative-type and reasoning fields so that every artifact has the same shape. Transient HTTP errors (429, 500, 502, 503) are retried with exponential backoff. This validation layer prevents a malformed or inconsistent answer from corrupting the aggregate results.

Output filenames are condition-tagged (`classification.<strategy>-<taxonomy>.json`) and written beside the shared, read-only `context.json`, so every condition reuses the same evidence and no condition overwrites another.

3.9 Evaluation Framework

Since Defects4J does not provide ground-truth ODC defect-type annotations, we use the post-fix classification as a fix-informed reference for evaluating the pre-fix prediction. The fix provides additional evidence about the nature of the defect, allowing

us to examine whether the classification inferred from symptoms and code context remains consistent when the actual repair is revealed.

3.9.1 The Reference Standard

Because no ODC labels exist for Defects4J, accuracy against ground truth cannot be measured for this dataset. We therefore measure agreement with the best available reference: the post-fix classification, produced by the same model, taxonomy, and prompt on the same bug, with the developer’s fix diff added to the evidence. Four considerations support this choice:

1. **The reference sees the evidence that defines the answer.** An ODC defect type describes the nature of the corrective change. The post-fix classification sees that change, while the pre-fix classification has to infer it.
2. **It follows the standard design when no ground truth exists.** The comparison is a two-rater design in which the second rater has more information, and Section 2.5 shows that agreement on ODC classification depends on the evidence available.
3. **Our data indicate that the post-fix classification is the more stable one.** The human-review flag is raised 16 times in the pre-fix classifications and never in the post-fix classifications, and the two strategies agree with each other more often in post-fix than in pre-fix mode, in every project studied.
4. **The fix isolates one fault among several.** A buggy Defects4J version rarely contains only the fault that the benchmark entry records: Callaghan and Fischer [4] identify on average 9.2 faults in each of the 311 versions of five Defects4J projects. The pre-fix evidence is drawn from code that may contain any of these faults, whereas the fix diff shows the correction of exactly one of them, the fault that the entry is about. The post-fix classification is therefore anchored to the same fault that the benchmark identifies, which the pre-fix evidence alone cannot guarantee (Section 4.4).

Agreement with this reference is not the same as correctness: both classifications can be wrong together, and Section 5.7 gives examples. We therefore interpret agreement as reproduction of a better-informed reference.

3.9.2 The Four-Tier Framework

We compare the pre-fix and post-fix classifications at four tiers, as shown in Figure 3.7.

1. **Tier 1 — Strict match:** Do the pre-fix and post-fix classifications assign the exact same ODC type? This is the most demanding metric.
2. **Tier 2 — Top-2 match:** Is the primary type of one classification found among the alternative types of the other? This captures cases where the model identified the type but ranked it second.
3. **Tier 3 — Family match:** Do both classifications fall within the same ODC family (Control and Data Flow or Structural)? This captures broad semantic agreement when the specific type differs.
4. **Tier 4 — Cohen’s κ :** A chance-corrected measure of strict agreement, which accounts for the agreement expected when most bugs fall into a few categories.

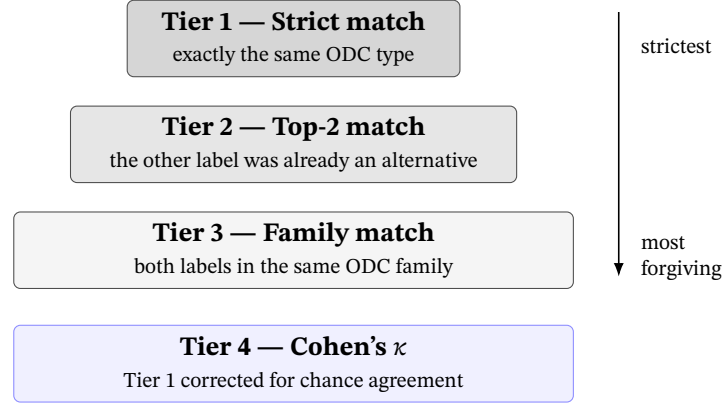


Figure 3.7: The four-tier evaluation framework. Tiers 1 to 3 progressively relax the agreement criterion; Tier 4 corrects Tier 1 for chance agreement.

Cohen’s κ is computed as

$$\kappa = \frac{p_o - p_e}{1 - p_e}, \quad (3.1)$$

where p_o is the observed agreement and p_e is the agreement expected by chance alone. It is read on the Landis–Koch scale, where 0.21–0.40 counts as fair, 0.41–0.60 as moderate, and $\kappa \geq 0.61$ as substantial agreement [21].

3.9.3 Reading Tiers 2 and 3

Tiers 2 and 3 produce high values that are close to their chance level. Because three types dominate the benchmark and all three belong to the same family, randomly shuffling the post-fix labels still yields top-2 and family match rates above 90%. Reported on their own, these two tiers therefore carry little information. We report them conditional on disagreement: among the bugs where Tier 1 fails, Tiers 2 and 3 describe how far apart the two labels are. Chapter 5 uses this reading throughout.

3.10 Summary

The proposed methodology combines three ideas: multi-modal evidence collected from Defects4J into a normalized case file, ODC’s principled taxonomy with an explicit escape category, and an enforced hypothesis-and-probe loop adapted from scientific debugging. Separating the taxonomy axis from the strategy axis gives a controlled condition space, and the dual-mode pre-fix/post-fix design provides an evaluation reference for a benchmark without labels. The next chapter describes the experimental setup, and Chapter 5 presents the results.

Chapter 4

Experimental Design and Data Collection

This chapter describes how we set up the experiments, what data we collected, how the pipeline was configured, and which statistical methods we used to analyse the results.

4.1 Dataset: Defects4J

The experiments use Defects4J v2.0, which provides over 830 active bugs across 17 open-source Java projects [17]. Table 4.1 lists the projects.

Each bug provides the following artifacts used by our pipeline: bug reports (from JIRA, GitHub, or project trackers), triggering test cases, buggy source code, fixed source code, and buggy-to-fixed diffs. Defects4J also provides command-line tools for checkout, compilation, test execution, and coverage analysis.

4.2 Experimental Setup

4.2.1 Evidence Collection Configuration

For each bug, the pipeline uses evidence in two modes:

- **Pre-fix:** Symptoms only (bug report, stack traces, failing tests, and code snippets), with no access to the fix.
- **Post-fix:** All pre-fix evidence plus the actual buggy-to-fixed code diff.

Table 4.1: Defects4J v2.0 projects and their approximate active bug counts. Per-project counts are version-dependent.

Project	Description	Active Bugs
Chart	JFreeChart plotting library	26
Cli	Apache Commons CLI	40
Closure	Google Closure Compiler	174
Codec	Apache Commons Codec	18
Collections	Apache Commons Collections	4
Compress	Apache Commons Compress	47
Csv	Apache Commons CSV	16
Gson	Google Gson	18
JacksonCore	Jackson JSON core	26
JacksonDatabind	Jackson data binding	112
JacksonXml	Jackson XML	6
Jsoup	HTML parser	93
JXPath	Apache JXPath	22
Lang	Apache Commons Lang	64
Math	Apache Commons Math	106
Mockito	Mocking framework	38
Time	Joda-Time	26
Total		836

Stack-frame filtering retains only project source frames, discarding JUnit, TestNG, Hamcrest, JDK-internal, and build-tool frames. Production snippets capture ± 12 lines around each suspicious frame; test snippets capture ± 18 lines and are limited to the first three failing tests per bug. Coverage analysis is optional and focuses on suspicious classes parsed from Cobertura XML.

Every `context.json` contains the bug ID, project name, and report URL; failing-test summaries, exception messages, and stack-trace frames; production-code and failing-test snippets; optional Cobertura coverage summaries; hidden-oracle fields (the modified classes and the optional fix diff); and the buggy/fixed revision metadata. The same `context.json` is read by every condition, so no condition sees different evidence. The context is read-only during classification, and the hidden-oracle fields are stored with the file but never enter the prompt, so the evaluation layer can check what a probe reached without the classifier having access to that information.

4.2.2 LLM Configuration

The classification uses the following configuration:

- **Provider and model:** Google Gemini API, model `gemini-3.1-flash-lite-preview`, with schema-enforced structured output. This model is used for every classification in the 410-bug study.
- **Second model:** OpenAI `gpt-5-mini`, accessed through an OpenAI-compatible API with its default temperature and reasoning effort, used for the cross-model analysis (Section 5.8) of the thirteen manually analysed bugs described in Section 5.7. Both models use the same evidence contexts and prompts.
- **Conditions:** *scientific-open* (default), *few-open*, and the *zero-free* baseline.
- **Reasoning loop:** At most six turns, with each probe’s returned payload truncated to 2,000 characters in the transcript.
- **Output format:** Structured JSON with the ODC type, alternative types, family, confidence score, reasoning trace, and, for the scientific condition, the full turn transcript.

Defects4J tooling is invoked through WSL for the collection stage, but because evidence is reused from the pre-collected corpus, the classification runs involve only LLM calls. Commands are scripted and dependency versions are pinned for reproducibility, and each run records per-classification turn and call counts. Each research question uses the conditions it requires, as listed in Table 4.2.

Table 4.2: Research questions, the conditions each uses, and the metrics that answer it

RQ	Conditions used	Metrics
RQ1	scientific-open (all four runs reported)	Type distribution; per-project variation
RQ2	All open-taxonomy runs	Escape rate; escape upper bound
RQ3	scientific-open, few-open	Four tiers: strict, top-2, family, κ ; drift transitions
RQ4a	zero-free, few-open, scientific-open	Distinct labels; entropy; vocabulary reduction; label reproducibility
RQ4b	few-open, scientific-open	Improved and degraded bugs; probe use; κ difference; transcripts of two models

4.2.3 Evaluation Configuration

Each bug produces a pre-fix and a post-fix classification per condition, and the pairs are compared using the four-tier framework of Section 3.9. In addition, the following

statistical methods are used in Chapter 5.

Monte Carlo permutation χ^2 test (RQ1). The standard χ^2 test of association assumes that every cell has an expected count of at least five. Our project-by-type table violates this assumption, because several ODC types are rare. We therefore shuffle the labels across bugs 10,000 times and compute the proportion of shuffled tables whose χ^2 statistic is at least as large as the observed one. We report Cramér’s V alongside the test as a measure of effect size.

Rule of three (RQ2). When zero events are observed in n trials, the 95% upper bound on the true event rate is approximately $3/n$. We use this bound to express the absence of escapes as a quantitative limit.

Bootstrap confidence intervals (RQ3). To quantify the uncertainty of strict match and Cohen’s κ , we resample bugs with replacement 2,000 times and report the resulting 95% percentile interval.

Paired bootstrap on the κ difference (RQ4b). To compare two conditions evaluated on the same bugs, we resample bugs 5,000 times, recompute the κ of both conditions on each resample, and take the difference. This preserves the pairing and yields a confidence interval on the difference itself.

McNemar’s exact test (RQ4a, RQ4b). For paired binary outcomes, such as whether each bug’s pre-fix label matches its post-fix label under two conditions, McNemar’s test compares the two discordant counts. Because it counts raw agreement without chance correction, we report it alongside κ .

Sign test (RQ4b). A non-parametric test of whether one condition outperforms another consistently across projects, regardless of the size of each difference.

Total variation distance and entropy (RQ1, RQ3, RQ4a). Total variation distance (TVD) measures how far apart two label distributions are, on a scale from 0 to 1. Shannon entropy measures how spread out a single distribution is, in bits. The vocabulary reduction ratio measures how strongly a taxonomy constrains the labels, and is computed as $1 - k/u$, where $k = 7$ is the number of ODC types and u is the number of distinct labels produced without a taxonomy.

4.3 Data Collection

From the benchmark, we pre-collected a shared evidence corpus of 410 pre-fix and 410 post-fix `context.json` payloads. Collecting this corpus took approximately one week, because each bug requires a checkout, a compilation, and a full test run. Every study run reuses this corpus, so no classification run triggers a fresh Defects4J checkout or test execution.

The results reported in Chapter 5 are based on those 410 bugs from six projects, shown in Table 4.3. The projects cover a range of software characteristics: a plotting library, a compiler, a general-purpose utility library, a numerical library, a mocking framework, and a date and time library. Closure, the largest project in Defects4J, contributes 153 of its 174 active bugs; the remaining Closure bugs were not collected or did not complete classification in all four configurations.

Table 4.3: The six projects in the study corpus

Project	Bugs	Character
Chart	26	Graphics and plotting; heavy object state
Closure	153	JavaScript compiler; program analysis, optimization, and type inference
Lang	61	String and utility functions; small, self-contained methods
Math	106	Numerical algorithms; computation-heavy
Mockito	38	Mocking framework; interface- and proxy-heavy
Time	26	Date and time arithmetic; boundary-condition heavy
Total	410	

Each of the 410 bugs was classified under two conditions (*scientific-open* and *few-open*) in two evidence modes, giving $410 \times 2 \times 2 = 1,640$ classifications, together with a *zero-free* baseline over the same 410 bugs in both modes for RQ4a.

All figures in Chapter 5 were recomputed directly from the per-bug classification artifacts using the pipeline’s own comparison functions. The analysis was first run on the five projects evaluated before Closure and reproduced the previously reported figures, and was then applied to the full 410-bug corpus. The per-project analysis workbooks were cross-checked against the raw artifacts; for Closure, all 612 labels in the workbook match the artifacts. Every classification artifact retains its full reasoning transcript, so each reported number can be traced back to individual model outputs.

4.4 Multi-Fault Context

Defects4J bugs do not exist in isolation. Each benchmark entry identifies a single fault, but Callaghan and Fischer [4] show, by transplanting the fault-revealing tests of later entries into earlier versions, that the buggy versions of five Defects4J projects contain on average 9.2 faults each. Their defects4j-mf dataset records, for every covered version, which other faults are still exposed in it. Our pipeline reads this dataset directly, without re-running the transplantation.

Of our 410 bugs, 372 belong to the five projects covered by defects4j-mf; Mockito is not covered. For 288 of these 372 bugs, the dataset records at least one other fault exposed in the same buggy version, with a mean of 8.0 other faults (median 6.0) in those versions. Among the thirteen scenario bugs of Section 5.7, the buggy versions of Math_17 and Math_23 contain eight other faults each, Lang_20 five, and Math_90 three.

Co-existing faults do not appear to drive the pre-fix/post-fix drift reported in Chapter 5: among these 288 bugs, those whose *scientific-open* label changes when the fix is visible have 8.2 other faults on average, and those whose label is stable have 8.0 (permutation test, 5,000 shuffles, $p = 0.77$).

Chapter 5

Results and Analysis

This chapter presents the results of our study for the four research questions. For each question, we describe the analysis, report the results and statistical tests, and discuss their interpretation and scope.

5.1 Classification Results Overview

The results are computed over the corpus summarized in Table 5.1. Throughout this chapter, *pre-fix* refers to classification from the buggy code, failing tests, and bug report only; *post-fix* refers to classification of the same bug with the developer’s fix diff added; and *drift* refers to a bug whose pre-fix and post-fix labels differ.

Table 5.1: The corpus used for all results in this chapter

Quantity	Value
Bugs	410: Chart 26, Closure 153, Lang 61, Math 106, Mockito 38, Time 26
Main conditions	<i>scientific-open</i> and <i>few-open</i>
Evidence modes	pre-fix and post-fix
Classifications	2 conditions $\times$ 2 modes $\times$ 410 bugs = 1,640
Ablation baseline	<i>zero-free</i> , over the same 410 bugs in both modes
Model	gemini-3.1-flash-lite-preview

Three conditions are compared. Under *zero-free*, the model describes the defect in its own words without a taxonomy. Under *few-open*, the model receives the seven ODC types and the “Other” category in a single prompt with definitions, a decision checklist, and worked examples. Under *scientific-open*, the model receives the same taxonomy and evidence but must complete the hypothesis–prediction–probe–observation

loop before answering. *scientific-open* is the default condition used for RQ1 and RQ3. The comparison between *few-open* and *scientific-open* isolates the contribution of the loop, because the two conditions differ only in the reasoning strategy.

5.2 RQ1: Defect-type Distribution

RQ1 characterizes the ODC composition of the benchmark and tests whether it varies across projects. If the defect-type profile is a property of the benchmark, a profile measured on one sample of projects generalizes; if each project has its own profile, aggregate statements about Defects4J depend on which projects dominate the sample. We report the pre-fix *scientific-open* classifications as the main result, together with the other three runs so that the effect of the strategy and evidence mode on the distribution is visible.

5.2.1 Results

Table 5.2 reports the distribution of ODC defect types under all four runs.

Table 5.2: RQ1: distribution of ODC defect types under the four runs (410 bugs each)

ODC type	Sci. pre-fix	Few pre-fix	Sci. post-fix	Few post-fix
Checking	198 (48.3%)	119	192	151
Algorithm/Method	175 (42.7%)	265	160	218
Assignment/Initialization	28 (6.8%)	11	37	28
Relationship	5	5	12	6
Function/Class/Object	3	8	4	2
Interface/O-O Messages	1	2	5	5
Timing/Serialization	0	0	0	0
Other	0	0	0	0
Total	410	410	410	410

Two observations stand out. First, Checking, Algorithm/Method, and Assignment/Initialization together account for 97.8% of the bugs under the default condition, and all three belong to the Control and Data Flow family. Second, Timing/Serialization does not appear in any of the 1,640 classifications. This is consistent with the construction of Defects4J, which consists of reproducible, single-threaded unit-test failures; a defect that manifests only under a race condition is unlikely to be reproducible enough to enter the benchmark.

Table 5.3 gives the per-project breakdown under the default condition.

Table 5.3: RQ1: per-project type counts, *scientific-open* pre-fix

Project	n	Algorithm	Checking	Assignment	Other types
Chart	26	5	15	6	0
Closure	153	69	77	2	5
Lang	61	22	33	5	1
Math	106	53	42	9	2
Mockito	38	17	16	4	1
Time	26	9	15	2	0

Closure differs from the other projects in one respect: Assignment/Initialization is almost absent (2 of 153 bugs, 1.3%), compared with 26 of 257 bugs (10.1%) in the other five projects. Closure’s defects fall almost entirely on the Algorithm/Method and Checking types.

5.2.2 Statistical Analysis

We tested whether the type distribution differs by project using the Monte Carlo permutation χ^2 test described in Section 4.2.3. Table 5.4 reports the results.

Table 5.4: RQ1: test of association between project and ODC type (10,000 permutations)

Condition	χ^2	dof	Monte Carlo p	Cramér’s V
Scientific pre-fix	36.11	25	0.096	0.133
Few-shot pre-fix	49.48	25	0.010	0.155

Under the default condition, the type distribution does not differ significantly across projects at the 5% level ($p = 0.096$), and the association is weak (Cramér’s $V = 0.13$). The evidence is weaker than for the five projects without Closure ($p = 0.381$), and the difference is driven mainly by Closure’s low share of Assignment/Initialization. Under the few-shot condition, the distribution does differ significantly by project ($p = 0.010$). This suggests that the single-call prompt is more sensitive to project-specific characteristics than the enforced loop, a point we return to in Section 5.5.2.

5.2.3 Discussion

These distributions are produced by an LLM rather than by human ODC raters, and so depend on the classification configuration. To quantify this dependence, we compare distributions using total variation distance. The distance between the scientific and few-shot pre-fix distributions is 0.23, whereas the distance between the pre-fix

and post-fix distributions within the scientific condition is only 0.05. The prompting strategy therefore changes the distribution more than the visibility of the fix does. For this reason, we report RQ1 for a stated condition and interpret it as the ODC profile produced by that configuration, not as a configuration-independent distribution of Defects4J. Within that scope, three Control and Data Flow types account for almost all of the bugs in these six projects, and the absence of Timing/Serialization reflects the nature of the benchmark rather than a property of ODC.

5.3 RQ2: Taxonomy Coverage

RQ2 asks whether any Defects4J bugs fall outside the seven ODC defect types. The open taxonomy makes this measurable: the prompt provides the seven types together with an explicit “Other” option, and instructs the model to select “Other” with a written justification for any bug that does not fit. We applied this to all 410 bugs under both strategies and both evidence modes, giving 1,640 opportunities to escape.

5.3.1 Results

Table 5.5 summarizes the results.

Table 5.5: RQ2: taxonomy coverage. The escape category was available in all 1,640 classifications and was never selected.

Observation	Count
Classifications selecting “Other” as the primary type	0 / 1,640
Classifications listing “Other” as an alternative type	1
Escape justification fields filled in	0
ODC types used	6 of 7
Human-review flags raised (all in pre-fix mode; none was an escape)	16 / 1,640

No classification selected the escape category. Closure, the compiler project and the project most likely to contain unusual defects, contributed 612 of these classifications without a single escape. Using the rule of three, the 95% upper bound on the true escape rate is 0.7% per condition ($n = 410$) and 0.2% when all 1,640 classifications are pooled.

5.3.2 Discussion

A zero escape count could reflect reluctance to use the “Other” category rather than genuine coverage. Two observations argue against this interpretation. First, the model did not restrict itself to the dominant types: it selected Relationship 28 times, Function/Class/Object 17 times, and Interface/O-O Messages 13 times across the corpus, so the full label space was in use. Second, when the taxonomy was removed in the *zero-free* condition (Section 5.5.1), the same model produced 366 distinct free-form labels for the same 410 bugs, showing that it is able to describe unusual defects in its own words.

The design space also includes a closed-taxonomy pass, in which the escape option is removed. The purpose of that pass is to measure whether removing the escape changes ordinary labels. Because the open taxonomy produced no escapes, a closed-taxonomy pass could only have measured the effect of prompt wording, and we did not run it.

The finding is therefore that the seven ODC types were sufficient for these six Defects4J projects: the escape category was available and was not used. This does not establish that the seven types are sufficient for all software, since Defects4J represents a specific kind of defect.

5.4 RQ3: Classification Accuracy

RQ3 evaluates how well the pipeline classifies a bug from pre-fix evidence, which is the situation at triage time before a fix exists. As described in Section 3.9, Defects4J provides no ODC labels, so we measure agreement between the pre-fix classification and the fix-informed post-fix reference using the four-tier framework.

5.4.1 Results

Table 5.6 reports the four tiers for both strategies.

As noted in Section 3.9.3, Tiers 2 and 3 are close to their chance level: randomly shuffling the post-fix labels still yields 92.5% top-2 and 92.9% family match for the scientific condition. Table 5.7 therefore reports these tiers over the bugs whose strict match failed.

The pre-fix *scientific-open* classification assigns the same defect type as the fix-informed reference for 70.2% of bugs. For a further 27.3%, the two labels differ

Table 5.6: RQ3: four-tier agreement between the pre-fix classification and the post-fix reference (410 bugs; 95% bootstrap intervals in parentheses)

Tier	Metric	Scientific	Few-shot
1	Strict type match	70.2% (65.6–74.9)	66.6% (62.0–71.2)
2	Top-2 match	97.1%	98.5%
3	Family match	94.1%	95.6%
4	Cohen’s κ	0.505 (0.433–0.574)	0.389 (0.310–0.468)

Table 5.7: RQ3: severity of disagreement, conditional on strict match having failed

Among bugs where pre-fix $\neq$ post-fix	Scientific (122 bugs)	Few-shot (137 bugs)
One label among the other classification’s alternatives	90.2%	95.6%
Both labels in the same ODC family	80.3%	86.9%
Unrelated (neither an alternative nor the same family)	10 bugs	2 bugs
as a share of all 410 bugs	2.4%	0.5%

but one of them appears among the other classification’s alternative types, or both belong to the same ODC family. For only 2.4% of bugs (10 of 410) is the pre-fix label unrelated to the reference.

A directional reading of the same disagreements is useful for practice. Asking only whether the pre-fix classification had *already recorded* the fix-informed type, the post-fix type appears among the pre-fix alternative types for 94 of the 122 scientific drifts (77.0%) and for 110 of the 137 few-shot drifts (80.3%). Shuffling the post-fix labels across the same alternative lists yields 36.6% and 39.0% respectively (permutation test, 5,000 shuffles, $p = 0.0002$), so this is not a consequence of three types dominating the corpus. Counting strict matches together with these bugs, the pre-fix classification names the fix-informed type either as its primary label or as a recorded alternative for 93.2% of bugs under the scientific condition and 93.4% under few-shot. Section 5.7.8 gives the reason this matters: where a defect admits more than one repair, the type of the repair that was not taken is usually the type already recorded as the alternative.

For Closure, the largest project in the corpus, disagreements are mostly near misses: 96.3% of its drifts have one label among the other classification’s alternatives, 88.9% remain within the same family, and only one Closure bug (Closure_171) is unrelated to the reference.

5.4.2 Statistical Analysis

Cohen’s κ of 0.505 indicates moderate agreement on the Landis–Koch scale, below the substantial band that begins at 0.61. The confidence intervals for the two strategies, [0.433, 0.574] and [0.310, 0.468], overlap only slightly.

To interpret these values, we measured the agreement between the two strategies when both are given the fix. With the fix visible, *scientific-open* and *few-open* agree on 76.3% of bugs ($\kappa = 0.614$); without the fix, they agree on 68.0% ($\kappa = 0.451$). This holds in all six projects. Two classifications by the same model, both with access to the fix, therefore still disagree on nearly a quarter of bugs. The pre-fix strict match of 70.2% lies about six percentage points below this level, which indicates that a substantial part of the remaining disagreement reflects ambiguity in the classification task rather than the absence of the fix.

5.4.3 Label Changes When the Fix Is Visible

Under the scientific condition, labels changed for 122 of 410 bugs (29.8%); under the few-shot condition, for 137 of 410 bugs (33.4%). For the five projects without Closure, the scientific drift rate is 26.5%, and for Closure alone it is 35.3%.

Although individual labels change, the overall distribution changes very little. The total variation distance between the pre-fix and post-fix distributions is 0.051 for the scientific condition and 0.129 for few-shot. Table 5.8 breaks down the 122 scientific drifts by type pair.

Table 5.8: RQ3: the three most frequent drift pairs, *scientific-open*. $A \rightarrow B$ counts bugs labelled A in pre-fix mode and B in post-fix mode.

Type A	Type B	Drifts	$A \rightarrow B$	$B \rightarrow A$
Algorithm/Method	Checking	75	38	37
Algorithm/Method	Assignment/Initialization	12	10	2
Checking	Assignment/Initialization	11	8	3

Of the 122 drifts, 98 (80.3%) occur among these three types, which are the types that human raters also confuse (Section 2.5). The Algorithm/Method–Checking boundary alone accounts for 75 drifts. For Closure, 43 of its 54 drifts cross this boundary, reflecting the nature of compiler defects, which are frequently described either as an analysis computing the wrong result or as an analysis failing to test a condition.

Part of this drift reflects what the two modes can see. A buggy Defects4J version usually contains several faults [4], so the pre-fix evidence may point at code or symptoms

of a fault other than the one the benchmark entry records, whereas the post-fix diff shows the repair of that one fault only. The drift is therefore partly a measure of the fix narrowing the evidence to a single fault and a single chosen repair (Section 5.7.8), rather than only a measure of classification error. The number of co-existing faults does not, however, predict which bugs drift (Section 4.4).

Two further patterns hold across all six projects:

1. **Algorithm/Method is the most frequent drift partner in every project, but the type it is confused with depends on the project.** It is confused mainly with Checking in Closure, Lang, Math, and Time; with Assignment/Initialization in Chart; and with a wider spread of types, including Interface/O-O Messages, in Mockito, which contains more interface-heavy code.
2. **The model’s confidence signals are not useful as a filter.** Across 1,640 classifications there were 16 human-review flags and no low-confidence classifications. In one case, a bug was flagged for review while reporting a confidence of 0.9; in another, the model issued four probes that did not reach the relevant code and still concluded with a confidence of 1.0.

5.4.4 Discussion

Agreement with the post-fix reference is not proof of correctness, because both classifications can be wrong in the same way. Our manual analysis (Section 5.7) includes two such bugs, Chart_17 and Math_90, which receive the same label in both modes and are both misclassified. The claim supported by RQ3 is therefore that pre-fix classification reproduces a better-informed reference, which is the property that matters at triage time, when the fix does not yet exist.

The two strategies also differ in how they disagree. The scientific condition achieves higher strict match and κ , but when it disagrees, the disagreement is more often severe: 8.2% of its drifts are unrelated to the reference, compared with 1.5% for few-shot. The few-shot condition more often lists the reference label as an alternative, whereas the scientific condition commits more firmly to a single type.

5.5 RQ4: Contribution of Each Component

Our pipeline adds two components to direct LLM classification: the ODC taxonomy and the enforced reasoning loop. RQ4a measures the contribution of the taxonomy by comparing *zero-free* with the two taxonomy-grounded conditions. RQ4b measures

the contribution of the loop by comparing *few-open* with *scientific-open*, which share the same taxonomy and differ only in the reasoning strategy. All comparisons use the same 410 bugs.

5.5.1 RQ4a: Taxonomy Grounding

Table 5.9 compares the three conditions on pre-fix labels.

Table 5.9: RQ4a: ablation ladder over 410 bugs. “Reproducible” is the share of bugs for which the model produced the identical label in both evidence modes.

Condition	Distinct labels	Entropy (bits)	Reproducible	κ
zero-free	366	8.376	15.1%	0.150
few-open	6	1.291	66.6%	0.389
scientific-open	6	1.446	70.2%	0.505

Without a taxonomy, the model produced 366 distinct labels for 410 bugs, and 342 of these labels were used exactly once. The most frequent free-form label, “Null Pointer Dereference”, covers 15 bugs. The vocabulary reduction ratio achieved by the seven-type taxonomy is 0.981. The free-form labels are also unstable: the model produced the identical label in both evidence modes for only 15.1% of bugs, and for only 9.2% of Closure bugs. Table 5.10 shows examples.

Table 5.10: RQ4a: free-form labels for four bugs under the *zero-free* condition. Each pair describes a similar defect, but no pair can be aggregated as the same label.

Bug	Free-form label, pre-fix	Free-form label, post-fix
Chart_9	Improper input validation logic	Incorrect boundary condition logic
Math_23	Logic error in optimization result selection	Logic error in optimization result tracking
Time_3	Incorrect state mutation during time arithmetic	Unnecessary state mutation during zero-value arithmetic
Chart_11	Incorrect state management in object comparison	Incorrect object reference in comparison logic

McNemar’s exact test on whether each bug’s pre-fix and post-fix labels match (free-form labels compared case-insensitively) gives 223 bugs consistent only under *scientific-open* against 18 consistent only under *zero-free* ($p = 3.8 \times 10^{-46}$), and 204 against 14 for *few-open* ($p = 2.1 \times 10^{-44}$).

This result is expected: labels chosen from a fixed set are more consistent than labels

written freely. Its role in this study is to establish that taxonomy grounding is a precondition for aggregate analysis. The free-form labels are often reasonable descriptions of the defect, but they cannot be counted or compared across bugs. Because this result validates the use of ODC rather than our pipeline design, the main question of RQ4 is the contribution of the reasoning loop, addressed next.

5.5.2 RQ4b: The Enforced Reasoning Loop

The comparison between *few-open* and *scientific-open* uses the same model, taxonomy, evidence, and bugs. The only difference is whether the model must complete the hypothesis–prediction–probe–observation loop before committing to a label. Because both strategies classified the same 410 bugs, we report the effect of the loop first as counts of bugs, and then as agreement statistics.

Bugs Improved and Degraded by the Loop

Each bug falls into one of four outcomes, depending on which strategy’s pre-fix label matches the reference. Table 5.11 counts them, first with each strategy scored against its own post-fix reference, as in RQ3, and then on the 313 bugs for which both post-fix classifications agree, so that both strategies are scored against the same answer.

Table 5.11: RQ4b: bugs improved and degraded by the enforced loop, relative to the few-shot prompt

Outcome of the pre-fix classification	410 bugs	313 consensus bugs
Both strategies match the reference	215	191
Improved: the loop matches, few-shot does not	73	43
Degraded: few-shot matches, the loop does not	58	33
Neither strategy matches	64	46
Net bugs gained by the loop	+15	+10
Strict match (few-shot → loop)	66.6% → 70.2%	71.6% → 74.8%

The loop improves the classification of 73 bugs and degrades 58, a net gain of 15 bugs, and the direction is the same against a shared answer (43 improved, 33 degraded). For 215 bugs both strategies are already right; the loop’s effect lies in the harder remainder of the corpus, where it resolves more bugs than it loses. Improvements outnumber or equal degradations in every project: Chart (8 improved, 2 degraded), Mockito (7, 2), Time (7, 4), Math (15, 14), Lang (9, 9), and Closure (27, 27).

Most of the changes lie on the boundary between Algorithm/Method and Checking. The few-shot prompt assigns Algorithm/Method to 265 of 410 bugs in pre-fix mode, the loop to 175. Thirty of the 73 improvements are bugs that few-shot labels Algorithm/Method and the loop correctly labels Checking, while the same move is wrong for 25 of the 58 degradations, so the loop corrects the few-shot prompt’s default more often than it overcorrects it.

Statistical Analysis

A paired bootstrap over the 410 bugs, with 5,000 resamples, gives a difference in Cohen’s κ of $\Delta\kappa = 0.115$ (0.389 for few-shot, 0.505 for the loop), with a 95% confidence interval of $[0.026, 0.208]$ and a two-sided $p \approx 0.012$, so the loop achieves significantly higher chance-corrected agreement with the reference. It has the higher κ in all six projects (sign test, one-sided $p = 0.016$), with the largest gains in Chart, Time, and Mockito, where few-shot is weakest; consistent with RQ1, the few-shot type distribution differs by project ($p = 0.010$) while the loop’s does not ($p = 0.096$). Three tests on raw counts do not reach significance: McNemar’s test on the 73 improved and 58 degraded bugs ($p = 0.221$), where few-shot’s concentration on Algorithm/Method raises its chance agreement; the same test on the consensus bugs (43 against 33, $p = 0.302$); and the Closure-only bootstrap ($\Delta\kappa = 0.065$, CI $[-0.108, 0.237]$).

Where the Gain Comes From: Held-Back Evidence

Both strategies start from the same evidence summary, but only the loop can request the evidence that the summary holds back, and only after committing to a prediction (Section 3.7). If this design is what helps, the gain should appear where the loop requests that evidence. Table 5.12 divides the improved and degraded bugs by how the pre-fix loop used its probes.

Table 5.12: RQ4b: improved and degraded bugs by the pre-fix loop’s use of held-back evidence

Pre-fix loop	Bugs	Improved	Degraded
Concluded without a probe	37	6	4
Issued one or two probes	263	42	41
Issued three or more probes	110	25	13
Retrieved the class modified by the fix	64	12	6

The loop’s advantage is concentrated where it investigates. With three or more probes

it improves 25 bugs and degrades 13, and when a probe retrieves the class that the developer later modified, it improves 12 and degrades 6, in both cases about two to one; with one or two probes the strategies are even (42 against 41). Withholding evidence is therefore useful in the way the design intends. Because the model decides how many probes to issue, and harder bugs attract more probes, this association does not prove that the probes cause the gain, but it locates the gain where the held-back evidence is used.

There is room for more. Thirty-seven pre-fix loops, and 126 post-fix loops, concluded without any probe, and the modified class is in the evidence store for only 125 of 410 bugs (13 of 153 for Closure). The measured improvement should therefore be read as a lower bound on what the loop can contribute with more complete evidence and fuller use of its probes. Section 5.9 examines, bug by bug and under two models, how the loop reaches its labels.

5.6 Why Types Diverge

Pre-fix and post-fix classifications are expected to diverge for a minority of bugs, for reasons intrinsic to semantic classification rather than pipeline error.

Evidence asymmetry. Pre-fix evidence shows what went wrong (the symptoms), while post-fix evidence shows what was changed (the fix), and one underlying defect can produce several symptoms. Human agreement on ODC classification shows the same dependence on evidence: Henningsson and Wohlin [11] report $\kappa = 0.16$ for raters working from fault descriptions, while Agnelo et al. [1] report $\kappa = 0.93$ for raters with full code and change context.

ODC boundary ambiguity. Some bugs legitimately sit near a type boundary, most often between Checking and Algorithm/Method, where a missing guard and a corrected computation describe the same defect from two perspectives. In our results, 80.3% of drifts fall within the three Control and Data Flow types, and 61% cross the Algorithm/Method–Checking boundary.

Co-existing faults. A buggy version usually contains faults other than the one the benchmark entry records [4], so the pre-fix code can show symptoms or suspicious constructs that the fix does not address. In our corpus, however, bugs whose label changes do not have more co-existing faults than bugs whose label is stable (Section 4.4), so this effect does not account for the drift we measure.

The four-tier framework makes this structure visible: the gap between strict match

and family match shows that disagreement is concentrated on boundary-adjacent types rather than distributed across unrelated families. At the level of aggregate statistics, which is the level at which ODC is designed to be used, the effect of the fix is small, with a total variation distance of 0.051 between the pre-fix and post-fix distributions.

5.7 Qualitative Analysis of Thirteen Bugs

The quantitative results show how often the pipeline agrees with its reference, but not how it reaches its labels. To examine this, we manually analysed thirteen bugs, reading the full evidence, the complete reasoning transcript including every probe and returned payload, and the developer’s fix.

The thirteen bugs were selected to cover every possible outcome of the pre-fix/post-fix comparison. Writing P for whether the pre-fix label matches the manual reading and Q for whether the post-fix label does, the scenarios are: both correct (S1); both wrong with the same label (S2); pre-fix correct and post-fix wrong (S3); pre-fix wrong and post-fix correct (S4); and both wrong with different labels (S5). A sixth group (S6) contains bugs selected to contrast the two strategies on identical evidence. Table 5.13 lists the bugs with all four labels and the manual reading.

Table 5.13: The thirteen manually analysed bugs. CHK = Checking, ALG = Algorithm/Method, ASN = Assignment/Initialization, INT = Interface/O-O Messages, REL = Relationship, FCO = Function/Class/Object.

Bug	Scenario	Scientific pre → post	Few-shot pre → post	Manual reading
Chart_9	S1	CHK → CHK	CHK → CHK	Checking
Lang_40	S1	ALG → ALG	ALG → ALG	Algorithm/Method
Chart_17	S2	CHK → CHK	CHK → ALG	Algorithm/Method
Math_90	S2	CHK → CHK	CHK → INT	Interface/O-O Msg.
Math_23	S3	ALG → ASN	ALG → ALG	Algorithm/Method
Math_17	S3	CHK → ALG	FCO → ALG	Checking
Time_3	S4	ALG → CHK	ALG → CHK	Checking
Chart_11	S4	ALG → ASN	ALG → ASN	Assignment/Init.
Time_27	S5	ALG → REL	ALG → CHK	Checking
Lang_20	S5	CHK → ALG	CHK → ALG	Assignment/Init.
Math_104	S6	ALG → ASN	ASN → ASN	Assignment/Init.
Mockito_26	S6	ASN → ASN	ALG → ASN	Assignment/Init.
Chart_7	S6	ASN → ALG	ALG → ASN	Assignment/Init.

5.7.1 S1: Both Modes Agree and Both Are Correct

Chart_9 shows the pipeline working as designed. The fix widens the predicate `endIndex < 0` to `(endIndex < 0) || (endIndex < startIndex)`, so an empty computed range is caught before it propagates. The loop hypothesised a missing guard, probed for the production class, and retrieved it, which occurred for only 64 of the 410 pre-fix loops. The symptom, hypothesis, retrieved evidence, and conclusion are consistent.

Lang_40 shows a correct classification of a non-Checking type: a case-insensitive search implemented with `toUpperCase` is replaced by an explicit `regionMatches` loop, and all four classifications correctly assign Algorithm/Method.

5.7.2 S2: Both Modes Agree and Both Are Wrong

This scenario shows that agreement between the two modes does not imply correctness.

In **Chart_17**, the developer reimplemented `clone()` so that it no longer delegates to `createCopy`; the guard on which the model focused is not changed by the fix. Both scientific classifications assign Checking with confidence 1.0, following the location where the exception surfaced rather than what was changed. We refer to this as **symptom-site bias**. Both scientific classifications concluded at the first turn without issuing a probe.

In **Math_90**, the bug report states that the problem “could be fixed by checking that the object is Comparable”, and this suggestion became the label, even in post-fix mode, where the diff shows an API signature change and the failing test expects a `ClassCastException` that the suggested guard would have prevented. We refer to this as **bug-report anchoring**.

In both bugs, the few-shot classification changed its label once the diff was visible, while the scientific classification did not. With gpt-5-mini, the scientific pre-fix classification of both bugs was also Checking, concluded at the first turn without a probe, which indicates that these two failure modes are not specific to one model; its post-fix classification corrected Chart_17 but not Math_90 (Section 5.8).

5.7.3 S3: The Fix Leads to a Worse Label

In **Math_23**, the fix threads a best point through every iteration of `BrentOptimizer`. The pre-fix scientific classification correctly assigns Algorithm/Method, but the post-

fix classification changes to Assignment/Initialization because the patch introduces a new variable with assignments. ODC classifies coordinated multi-site assignment changes as Algorithm/Method. The few-shot classification did not change.

In **Math_17**, a missing range check is added, and its `else` branch routes out-of-range values to the general multiplication instead of the fast path. Under ODC, a Checking defect remains Checking even when the fix adds a branch, and the pre-fix classification applied this rule correctly, while the post-fix classification changed its label based on the added branch, although its own alternative-types note stated that “a check is added”. This bug is also the one case in which the developers made two repairs of different types at once (Section 5.7.8), so a single-label reference cannot represent the fix completely.

In both cases, the post-fix classification followed the surface form of the patch rather than the taxonomy’s definition. This is a limitation of the post-fix reference, and it is one reason RQ3 is seen as reproduction of a reference rather than as correctness.

5.7.4 S4: The Fix Leads to a Better Label

Time_3 is a case of a correct mechanism with an incorrect label. The pre-fix classification described the exact fix, stating that the methods “do not contain an `if (amount == 0) return; guard`”, but assigned Algorithm/Method, because it classified according to its explanation of the root cause rather than the nature of the change. The post-fix classification corrected the label to Checking, and both strategies changed in the same way. The developer added the guard to each add method rather than changing the chronology arithmetic itself, which is a repair choice we return to in Section 5.7.8.

Chart_11 is a one-token copy-paste error: `p1.getPathIterator` should be `p2.getPathIterator`. The pre-fix loop used five turns and four probes, never reached the modified class because it is not present in `context.json`, and still concluded with confidence 1.0. Unsuccessful probes did not reduce the model’s confidence.

5.7.5 S5: Both Modes Wrong, with Different Labels

In **Time_27**, the pre-fix classification hypothesised an integer overflow while also stating that the value fits in an `int`, and did not revise the hypothesis. The post-fix classification interpreted an added guard as a structural Relationship defect.

Lang_20 illustrates a limitation of the task itself. A null dereference occurs inside a `StringBuilder` capacity estimate. A null guard would be a Checking fix, but the

developer instead changed the initial capacity expression so that the dereference no longer occurs, which is an Assignment/Initialization fix. Because ODC classifies the nature of the correction, a bug with more than one reasonable fix can have more than one valid defect type, and a pre-fix classification cannot determine which fix the developer will choose. Section 5.7.8 collects the bugs in which we observed this.

5.7.6 S6: Scientific and Few-Shot on Identical Evidence

In **Math_104**, the fix changes a single constant, `DEFAULT_EPSILON`, from `10e-9` to `10e-15`. The scientific classification’s first prediction named both the constant and the new value, but assigned Algorithm/Method because its hypothesis concerned the convergence algorithm. The few-shot prompt’s contrastive guidance, which distinguishes a wrong threshold value from wrong logic, led to the correct label.

In **Mockito_26**, `primitiveValues.put(double.class, 0)` autoboxes to `Integer`, and the fix writes `0D`. The scientific classification was correct in both modes. The correct label followed from how the hypothesis was phrased (a wrong value) rather than from the probes: four pre-fix probes returned no new information, and the modified class was not in the evidence store.

In **Chart_7**, the scientific classification changed from Assignment/Initialization to Algorithm/Method, while the few-shot classification changed in the opposite direction on identical evidence. The scientific pre-fix label is correct, but its stated mechanism, a “stale cached index”, is not supported by the code; the actual defect is the use of `minMiddleIndex` where `maxMiddleIndex` was intended. A label-only metric scores this classification as correct.

5.7.7 Strategies Against the Manual Reading

With Gemini, the scientific condition matches the manual reading for 6 of 13 bugs in pre-fix mode and the few-shot condition for 4, consistent with the RQ4b result that the loop improves pre-fix classification. In post-fix mode the order reverses (6 against 11), because the scientific classification kept its own line of reasoning and made little use of the fix diff, as in **Chart_17** and **Math_90**. Section 5.8 examines whether this depends on the model (Table 5.16), and Section 5.9 studies the individual bugs on which the two strategies differ. Because these bugs were selected to illustrate specific scenarios, this comparison describes mechanisms rather than measuring accuracy.

5.7.8 When the Developer Had More Than One Option

A further reading of the fixes behind the thirteen bugs shows a limit that is not a property of the pipeline but of the reference itself. An ODC defect type describes the nature of the correction that was made, so the post-fix label records *the repair the developer chose*, not the repair the defect required. Where more than one repair was available, the developer’s choice, which may be driven by risk, minimality, or the design of the surrounding code, decides the defect type that the post-fix classification can observe. Table 5.14 lists the four bugs in which we found this effect.

Table 5.14: Bugs whose ODC defect type depended on which repair the developer chose. Types in parentheses are those of the manual reading; abbreviations as in Table 5.13.

Bug	Repair that was available	Repair the developer made
Math_90	a Comparable check before the value is stored, as the bug report itself suggests (CHK)	a new add Value(Comparable<?>) overload, with the deprecated add Value(Object) delegating to it through a cast (INT)
Math_17	a range check on the fast path alone (CHK)	both: a range guard <i>and</i> an alternative path that routes out-of-range values to the general multiplication (CHK and ALG)
Time_3	a more accurate computation inside the chronology arithmetic (ALG)	an amount != 0 guard added to each add method, leaving the arithmetic untouched (CHK)
Lang_20	a null check before the dereference (CHK)	a rewritten capacity expression, noOfItems * 16, so the dereference no longer occurs (ASN)

Three effects appear in this small set. In Math_90 and Lang_20, a Checking repair was available and was not taken: the Math_90 fix changes the method signature so that the type constraint is expressed by the API rather than by a guard, and the Lang_20 fix removes the dereference instead of guarding it. In both, the recorded defect type is the one the implemented repair belongs to rather than the one the symptom suggests. In Time_3, the developer chose the safer of two correct repairs and left the underlying computation untouched, so the recorded type reflects an engineering judgement about risk as much as the defect. Math_17 is different again: the developers made both a check and an algorithmic change, while our reference assigns one type per bug and can therefore record only one of them.

The multi-fault view of Defects4J [4] places these choices in context. A fix in Defects4J repairs one fault in a version that usually contains others: according to defects4j-mf,

the buggy versions of Math_17 and Lang_20 contain eight and five other exposed faults, and that of Math_90 three (Section 4.4). The developer’s commit is therefore one targeted repair among several possible changes to the same code, and the post-fix reference records the type of that repair. This is what a fix-informed reference should record, since it identifies the fault the benchmark is about, but it also means that a pre-fix classification, which reads the whole buggy version, has more than one reasonable answer available.

What the recorded alternatives contribute. Our classification schema never commits to a single type. Every classification also returns one or two close alternative types, each with a written reason for not being the primary label (Section 3.5), and Tier 2 of the evaluation framework is defined over exactly these lists (Section 3.9). The repair-choice cases show why that design is necessary rather than merely convenient: when a defect admits more than one repair, and the buggy version contains other faults that a pre-fix reading may also pick up [4], the type of the repair that was *not* taken is usually the type the classification has already recorded as its alternative. Across the 122 bugs whose label changes once the fix is visible, the post-fix type appears among the pre-fix alternatives for 94 of them (77.0%), against 36.6% expected if the post-fix labels were shuffled across the same alternative lists (permutation test, 5,000 shuffles, $p = 0.0002$). Counting these together with the strict matches, the pre-fix classification names the fix-informed type either as its primary label or as a recorded alternative for 382 of 410 bugs (93.2%).

This does not resolve the ambiguity, and it is not a claim that the pipeline anticipates which repair a developer will choose. The classification reports the type the pre-fix evidence favours and preserves the competing type instead of discarding it, so a reader of the output can see the other repair that was open. The limit is visible in the same set of bugs: the pre-fix alternatives for Math_90 are Algorithm/Method and Assignment/Initialization, and the interface-level repair the developers made is not among them. Where several repairs are viable, a recorded alternative with a stated reason is more useful for triage than a single label, and it is the form in which our results should be read.

These cases sharpen the interpretation of RQ3. A pre-fix classification cannot know which repair a developer will select, so for bugs of this kind no pre-fix label can be expected to reproduce the post-fix reference, however good the reasoning is. They also bound the reference itself: the post-fix classification is better informed than the pre-fix one, but it is better informed about the repair, not about a defect type that exists independently of it. This is the concrete form of the caveat stated in Section 2.5

and revisited in Chapter 6, and it is a further reason to read RQ3 as reproduction of a reference rather than as correctness.

5.7.9 Findings from the Manual Analysis

Reading the reasoning transcripts, rather than only the labels, yields the following findings:

1. **Agreement is not correctness.** In S2, both scientific classifications agree with confidence 1.0 and are wrong.
2. **The fix can improve or worsen a label.** In S4 it corrects the label; in S3 it leads the model to follow the surface form of the patch instead of ODC's definition.
3. **The label and the reasoning can diverge.** Chart_7 is correct for an unsupported reason, while Time_3 and Math_104 identify the right mechanism but assign the wrong label. A label-only metric cannot detect either case.
4. **Confidence is not a reliable signal.** Chart_11 issued four unsuccessful probes and reported confidence 1.0.
5. **Some pre-fix labels are underdetermined.** Lang_20 has two reasonable fixes corresponding to two different ODC types.
6. **The loop is not always exercised.** Chart_17 and Math_90 concluded at the first turn without probes; in Math_90, a single probe of the failing test would have contradicted the hypothesis.
7. **The reference records the repair, not the defect alone.** In four bugs (Section 5.7.8) another repair of a different ODC type was available, and in one of them the developers made two repairs at once, so the post-fix label reflects the developer's choice as much as the defect. Each of these fixes repairs one fault in a version that also contains others [4].

This analysis is a qualitative study of mechanisms and not an accuracy measurement. The thirteen bugs were selected to populate specific scenarios, none of them is from Closure, and the reference reading is the judgement of a single rater applying the pipeline's ODC definitions. Three of the bugs, Time_27, Lang_20, and Math_17, remain open to reasonable disagreement.

5.8 Cross-Model Analysis

The pipeline is model-independent: the evidence contexts, prompts, conditions, and validation layer do not change with the LLM that performs the classification. We therefore ran the pipeline with two models, Google gemini-3.1-flash-lite-preview and OpenAI gpt-5-mini, and compared their classifications of the thirteen scenario bugs of Section 5.7. Because these bugs cover every outcome of the pre-fix/post-fix comparison and have a manual reading, they allow each classification to be scored and each reasoning transcript to be inspected. Comparing the two models on the same bugs, evidence, and prompts separates behaviours that arise from the pipeline, which appear under both models, from behaviours that arise from the model chosen.

Table 5.15 lists the labels produced by both models in all four configurations, and Table 5.16 summarizes their agreement with the manual reading.

Table 5.15: Labels assigned by both models to the thirteen scenario bugs. S = *scientific-open*, F = *few-open*; labels matching the manual reading are shown in bold. Abbreviations as in Table 5.13.

Bug	Manual	Gemini				gpt-5-mini			
		S-pre	S-post	F-pre	F-post	S-pre	S-post	F-pre	F-post
Chart_9	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK	CHK
Lang_40	ALG	ALG	ALG	ALG	ALG	ALG	ALG	ALG	ALG
Chart_17	ALG	CHK	CHK	CHK	ALG	CHK	ALG	CHK	ALG
Math_90	INT	CHK	CHK	CHK	INT	CHK	CHK	CHK	INT
Math_23	ALG	ALG	ASN	ALG	ALG	CHK	ALG	CHK	ALG
Math_17	CHK	CHK	ALG	FCO	ALG	ALG	CHK	ALG	CHK
Time_3	CHK	ALG	CHK	ALG	CHK	CHK	CHK	ALG	CHK
Chart_11	ASN	ALG	ASN	ALG	ASN	ASN	ASN	ALG	ASN
Time_27	CHK	ALG	REL	ALG	CHK	ALG	CHK	ALG	CHK
Lang_20	ASN	CHK	ALG	CHK	ALG	CHK	CHK	CHK	ASN
Math_104	ASN	ALG	ASN	ASN	ASN	ASN	ASN	ASN	ASN
Mockito_26	ASN	ASN	ASN	ALG	ASN	ASN	ASN	ASN	ASN
Chart_7	ASN	ASN	ALG	ALG	ASN	ALG	ASN	ALG	ALG

5.8.1 Behaviours of the Pipeline

Looking at the classifications of the thirteen scenario bugs from both models, several behaviours appear regardless of the model, which indicates that they arise from the pipeline, the evidence, or the classification task.

1. **The enforced loop outperforms the few-shot prompt in pre-fix mode.** For

Table 5.16: Agreement with the manual reading and reasoning behaviour of both models on the thirteen scenario bugs

Measure	Gemini	gpt-5-mini
Match with manual reading, scientific pre-fix	6 / 13	6 / 13
Match with manual reading, few-shot pre-fix	4 / 13	4 / 13
Match with manual reading, scientific post-fix	6 / 13	11 / 13
Match with manual reading, few-shot post-fix	11 / 13	12 / 13
Scientific pre-fix/post-fix drift	8	5
Few-shot pre-fix/post-fix drift	9	8
Mean probes per scientific run, pre-fix / post-fix	1.7 / 0.6	3.0 / 1.9
Scientific pre-fix runs reaching the six-turn limit	0	4

both models, the scientific condition matches the manual reading for 6 of 13 bugs and the few-shot condition for 4.

2. **The fix diff resolves most ambiguity.** With the fix visible, the few-shot condition matches the manual reading for 11 of 13 bugs with Gemini and 12 of 13 with gpt-5-mini, which supports the use of the post-fix classification as the evaluation reference.
3. **Symptom-site bias and bug-report anchoring come from the evidence.** Both models assign Checking to Chart_17 and Math_90 in pre-fix mode, concluding at the first turn without a probe. The misleading cues, an exception raised by an unchanged guard and a fix suggested in the bug report, are part of the pre-fix evidence, so both models follow them.
4. **Some pre-fix labels are underdetermined by the task.** Neither model’s scientific classification recovers the developer’s Assignment/Initialization fix for Lang_20, whose symptom is equally consistent with a missing null check.

5.8.2 Behaviours of the Model

Other behaviours differ between the two models, and therefore depend on the model chosen rather than on the pipeline.

1. **Use of the fix diff within the loop.** Gemini’s scientific post-fix classification

matches the manual reading for 6 of 13 bugs, while gpt-5-mini’s matches 11. The limited use of the fix diff by the scientific condition, observed in Section 5.7.7, is therefore a property of the main model rather than of the loop.

2. **Depth of investigation.** gpt-5-mini issues almost twice as many probes per scientific run (3.0 against 1.7), reaches the six-turn limit on four pre-fix runs, and changes its scientific label less often between the two evidence modes (5 drifts against 8); Section 5.9 examines why.
3. **Which bugs are classified correctly.** Although the two models reach the same pre-fix totals, they share only 3 of their 6 correct scientific pre-fix labels. The model therefore determines which ambiguous bugs are resolved, even when the overall rate is unchanged.
4. **Sensitivity of each strategy to the model.** The two models assign the same label in 10 of 13 bugs under the few-shot condition in each evidence mode, but in only 7 of 13 under the scientific condition. Because the loop lets each model choose which evidence to request, its outcome depends more on the model than the outcome of a single static prompt does.
5. **Output-format compliance.** gpt-5-mini omitted the confidence field in 18 of its 52 classifications, because the OpenAI-compatible interface does not enforce the output schema as Gemini’s structured output does. Model-reported confidence is therefore not comparable across models.

We also examined the effect of gpt-5-mini’s reasoning effort setting. On four scenario bugs in the few-shot pre-fix configuration, the default setting matched the manual reading for three bugs and the high setting for two, so increasing reasoning effort did not improve classification.

Taken together, the comparison shows that the main conclusions about the pipeline, the advantage of the loop in pre-fix mode, the value of the fix as a reference, and the evidence-driven failure modes, hold across both models, while the loop’s use of the fix diff and its depth of investigation depend on the model.

5.9 A Closer Study of the Scientific Loop

The central design choice of this thesis is the enforced loop, and RQ4b shows that it improves pre-fix classification over a strong few-shot prompt. This section studies how it does so. For each of the thirteen scenario bugs, we compare the loop with the few-shot prompt on identical evidence under both models, and read the transcripts

to identify where the loop leads to a better label, where it leads to a worse one, and which of these behaviours come from the pipeline rather than from the model. The findings are learning points for the design of the loop, and they motivate the future work in Section 7.3.

5.9.1 Where the Loop and the Few-Shot Prompt Differ

Table 5.17 lists every bug for which exactly one of the two strategies matches the manual reading.

Table 5.17: Scenario bugs on which exactly one strategy matches the manual reading, on identical evidence

Mode	Model	Only the loop is right	Only few-shot is right
Pre-fix	Gemini	Math_17, Mockito_26, Chart_7 (3)	Math_104 (1)
Pre-fix	gpt-5-mini	Time_3, Chart_11 (2)	none (0)
Post-fix	Gemini	none (0)	Chart_17, Math_90, Math_23, Time_27, Chart_7 (5)
Post-fix	gpt-5-mini	Chart_7 (1)	Math_90, Lang_20 (2)

In pre-fix mode, which is the setting the pipeline is designed for, the loop is the better strategy under both models: it alone is right for five bug classifications and the few-shot prompt alone for one. In post-fix mode, the few-shot prompt is the better strategy, particularly with Gemini. The two modes therefore call for different strategies, a point we return to below.

5.9.2 Where the Loop Does Better

Time_3 (gpt-5-mini). Every classification of this bug describes the same mechanism: the add methods do not return early when the amount is zero. The few-shot classification nevertheless assigns Algorithm/Method, and Gemini’s few-shot reasoning explicitly states that the defect is “not a missing guard”. The gpt-5-mini loop restated the hypothesis as a testable claim on each of its six turns, ending with a missing zero-amount short-circuit or guard as the mechanism, and concluded Checking, which is the type of the developer’s fix.

Chart_11 (gpt-5-mini). The class under test is not in the evidence store, so no probe could return its source. Across four turns, the loop’s prediction narrowed from a general claim that one path iterator serves both paths to a specific one: an incorrect as-

signment in which the second iterator is obtained by calling `getPathIterator` on the first path. It concluded Assignment/Initialization. The fix changes exactly this assignment, `iterator2 = p1.getPathIterator(null)` becomes `iterator2 = p2.getPathIterator(null)`. The few-shot classification described the same symptom as “wrong iterator usage” and assigned Algorithm/Method.

Math_17 (Gemini). The bug report says that `multiply(int)` supports only a limited range while the interface requires any integer. The few-shot classification followed this wording and assigned Function/Class/Object, a design-level gap. The loop’s hypothesis had to name a concrete mechanism, a range check that rejects values of 10,000 or more, and its prediction located that check in the method, which led to Checking.

These cases share one pattern. The loop does not win only by retrieving hidden code; it also wins because it must state a specific, falsifiable mechanism and predict what the code should contain. Writing that prediction moves the model from a general description of the symptom (“wrong iterator usage”, “limited range”) to the concrete change a developer would make (a wrong assignment, a missing check), and the ODC type follows from the change. It does not guarantee sound reasoning: in Chart_7 the loop’s label is right but its stated mechanism is not (Section 5.7).

5.9.3 Where the Few-Shot Prompt Does Better

Math_104 (Gemini, pre-fix). The loop’s prediction named both the constant and its value, $10e-9$, but its hypothesis was phrased around the convergence algorithm, and it concluded Algorithm/Method. The few-shot prompt includes contrastive guidance that separates a wrong threshold value from wrong logic, and it assigned Assignment/Initialization. The loop found the right mechanism and chose the wrong label.

Post-fix mode. With the fix diff visible, the few-shot prompt is the better strategy for Gemini (11 against 6 of 13): its loop kept a symptom-based conclusion for Chart_17 and Math_90 without probing, and followed the surface of the patch in Math_23. gpt-5-mini’s loop used the diff far better (11 of 13), losing to few-shot only on Math_90 and Lang_20, the two bugs whose fix was one of several possible repairs (Section 5.7.8).

5.9.4 Why Gemini Stops Early and gpt-5-mini Keeps Investigating

On the thirteen pre-fix loops, Gemini issued 22 probes over 35 turns and gpt-5-mini 39 probes over 52 turns, reaching the six-turn limit four times; a loop took 7 seconds

per bug with Gemini and 92 with gpt-5-mini.

Gemini’s early stops follow a consistent pattern. The loop instructions ask the model to conclude as soon as the evidence supports one type, and Gemini often treats the initial summary as enough: across the corpus, 31 of its 37 pre-fix loops without a probe concluded with confidence 1.0, and 19 of them justify stopping by describing the evidence in the summary as clear. In post-fix mode, where the diff is part of the summary, 122 of the 126 loops without a probe concluded with confidence 1.0. gpt-5-mini, a reasoning model, keeps its hypothesis open and continues testing even when the probes return little.

What gpt-5-mini gains from the extra turns is not mainly more code. Thirty-five of its 39 pre-fix probes requested the evidence inventory, 16 were refused as repeats, and it retrieved the modified class for one bug, against two for Gemini, whose probes were targeted (20 of 22 requested a snippet). Its benefit comes from more rounds of hypothesis and prediction, each forcing a more specific mechanism, as in Time_3 and Chart_11. The loop thus contributes through the evidence it retrieves (Section 5.5.2) and through the discipline of stating and revising a prediction, and the model decides how much of each a run obtains.

5.9.5 Lessons for the Scientific Loop

The study yields four learning points for the next version of the loop.

1. **The loop pays off when it is used.** Its gain is concentrated in runs that probe several times or retrieve the modified code (Table 5.12). Requiring at least one probe before a pre-fix conclusion, instead of asking the model to conclude as soon as possible, would move more runs into this regime.
2. **A right mechanism can still receive the wrong type.** In Math_104, and in Time_3 under Gemini, the conclusion step lacked the contrastive guidance between neighbouring types that the few-shot prompt provides; adding it would combine the strengths of both strategies.
3. **In post-fix mode the hypothesis should be tested against the fix.** A required comparison with the changed lines would stop the loop from keeping a symptom-based conclusion that the diff contradicts.
4. **Neither strategy is best everywhere.** The loop is better from symptoms, the few-shot prompt when the fix is visible, and the two are even when the loop does not investigate, so a pipeline that chooses its strategy from the evidence

available could take the better result in each case.

5.10 Summary

Table 5.18 summarizes the findings for each research question.

Table 5.18: Summary of findings for the four research questions

RQ	Finding
RQ1	Three ODC types cover 97.8% of 410 bugs. Under the default condition, the type distribution does not differ significantly across the six projects (Monte Carlo $p = 0.096$, Cramér’s $V = 0.13$), although Closure has almost no Assignment/Initialization defects
RQ2	The “Other” category was available in 1,640 classifications and selected in none. The 95% upper bound on the true escape rate is 0.2%
RQ3	Pre-fix classification reproduces the fix-informed reference for 70.2% of bugs ($\kappa = 0.505$); only 2.4% of pre-fix labels are unrelated to the reference, and most disagreements are near misses. 29.8% of labels change when the fix becomes visible, but 80.3% of the changes remain within three closely related types, and the aggregate distribution changes by a total variation distance of only 0.051.
RQ4a	Without a taxonomy, the model produces 366 distinct labels for 410 bugs and reproduces its own label in both modes for only 15.1% of bugs; taxonomy grounding is a precondition for aggregate analysis
RQ4b	The enforced loop improves the classification of 73 bugs and degrades 58, a net gain of 15, and significantly improves chance-corrected agreement ($\Delta\kappa = 0.115$, 95% CI [0.026, 0.208]), with the higher κ in all six projects. The gain is concentrated where the loop uses held-back evidence: 25 improved against 13 degraded when it issues three or more probes. Its gain on Closure alone is not significant
Cross-model	On the scenario bugs, both Gemini and gpt-5-mini show the pre-fix advantage of the scientific loop (6 versus 4 of 13; the loop alone is right five times, few-shot alone once) and the same evidence-driven failure modes. How long the loop is kept open depends on the model (1.7 versus 3.0 probes per run), and so does its use of the fix diff (6 versus 11 of 13)

Chapter 6

Threats to Validity

6.1 Internal Validity

LLM output variability. LLM classification is non-deterministic. We mitigate this with fixed prompt templates, low-temperature generation where supported, schema-enforced structured output, and a strict post-generation validation layer. The pipeline supports multi-run aggregation, but we did not classify each bug repeatedly, so we cannot report run-to-run variance.

Pre-fix evidence purity. Two channels can leak fix knowledge into the pre-fix classification if left unsanitised: `defects4j info` output lists the classes touched by the fix commit, and tracker reports for resolved issues often contain post-fix discussion. Both are removed from the pre-fix payload at classification time (Section 3.3), while the post-fix payload is left unchanged.

Instrument-prior revision. Development prompts stated that most Defects4J bugs fall into three named ODC types, which risked biasing the distribution measured in RQ1. This statement was replaced with a neutral anti-default rule that names no expected distribution, and every classification reported in Chapter 5 uses the revised prompt.

6.2 Construct Validity

Agreement as a proxy for correctness. Pre-fix/post-fix agreement is a proxy for correctness, not a direct measure of it, since Defects4J provides no ODC labels. The

manual analysis includes bugs that receive the same incorrect label in both modes. This strategy is justified by ODC’s two-phase model and by the evidence dependence of human agreement (Section 2.5), but it cannot substitute for expert-labelled validation.

The reference records one chosen repair. Because an ODC defect type describes the correction, the post-fix reference captures the repair the developer implemented, for one of the several faults that a buggy Defects4J version typically contains [4]. Section 5.7.8 documents four bugs in which a repair of a different ODC type was equally available, and one in which the developers made two repairs of different types while the reference records a single label. For such bugs the pre-fix label cannot be expected to reproduce the reference, and the disagreement measures the space of valid repairs rather than a classification error. This is the reason every classification also records close alternative types with a written justification: for 77.0% of the bugs whose label changes, the fix-informed type is already present in the pre-fix alternatives, so the competing repair remains visible in the output even when the primary label does not match.

Intended use of ODC. ODC was designed to provide process feedback from distributions of defect types, not to route individual bugs to developers. Our aggregate-level results support the former use; we make no claim about the latter.

6.3 External Validity

Partial benchmark coverage. The results cover 410 bugs from six of the 17 Defects4J projects, including 153 of the 174 Closure bugs. The addition of Closure lowered agreement and changed the per-project pattern of the loop’s gain, so the results may shift further when the remaining projects are included.

Model dependence. The corpus-level analysis uses `gemini-3.1-flash-lite-preview`, and the cross-model analysis uses both `gemini-3.1-flash-lite-preview` and `gpt-5-mini` on the scenario bugs (Section 5.8). The cross-model analysis shows that some behaviours, such as the use of the fix diff within the loop, depend on the model, so corpus-level rates may differ for other models.

Language and dataset. The pipeline is designed for Java and Defects4J. Porting it to other datasets such as Bears or BugSwarm, or to other languages, would require

new source resolution and trace extraction, although the classification logic itself is language-independent.

Multi-fault programs. Callaghan and Fischer [4] identify on average 9.2 faults in each buggy version of five Defects4J projects, and defects4j-mf records at least one other exposed fault for 288 of our 372 bugs from those projects. In pre-fix mode the model may therefore read code affected by several faults, while the post-fix diff concerns only one. We checked whether this explains the drift and found no association (8.2 against 8.0 other faults for bugs with and without a label change, $p = 0.77$), but defects4j-mf does not cover Mockito, and the effect of co-existing faults on individual classifications cannot be excluded.

6.4 Other Threats

Data contamination. Defects4J bugs and their fixes are public, and the model may have encountered them during pre-training. We mitigate this by evaluating pre-fix/post-fix consistency rather than absolute correctness, and by retaining full reasoning traces for human review.

Confidence fields. The model’s confidence and human-review fields do not provide a usable signal (Section 5.4.3), so any future filtering or escalation mechanism must rely on other information.

Chapter 7

Conclusion

This chapter summarizes our work, discusses its limitations, and outlines future research directions.

7.1 Summary

This thesis presents an automated, evidence-grounded pipeline that classifies Defects4J bugs into ODC Design/Code defect types using an LLM-driven scientific classification approach: an enforced loop of hypothesis, prediction, probe, and observation, inspired by scientific debugging and adapted to classification rather than localization. The key contributions are:

1. A two-variable condition space that separates the label space from the reasoning strategy, giving five controlled, comparable conditions.
2. An open-set ODC formulation with a justified escape category, which turns taxonomy coverage into a measured quantity.
3. An enforced scientific reasoning loop in which the model must commit to a hypothesis and probe held-back evidence before concluding, leaving an auditable transcript.
4. A dual-mode pre-fix/post-fix evaluation with a four-tier framework, which provides an evaluation reference for a benchmark without ODC labels.
5. A bug-level study of the scientific loop against a few-shot prompt under two LLMs, which identifies where the loop helps, where it falls short, and which behaviours belong to the pipeline rather than to the model.

Together, these address the five limitations of prior work set out in Section 1.3: coverage through scalable automation, semantic defect-type labels rather than structural or patch-level ones, classification from pre-fix evidence, controlled oracle leakage, and an evaluation framework that does not require ground-truth labels.

Across 410 bugs from six projects and 1,640 classifications, three ODC types account for 97.8% of the bugs, with no significant variation across projects under the default condition. The “Other” category was never selected, bounding the true escape rate at 0.2%. A classification made before the fix exists reproduces the fix-informed reference for 70.2% of bugs, and is unrelated to it for only 2.4%. The enforced loop improves the classification of 73 bugs and degrades 58 compared with a strong few-shot prompt, significantly improves chance-corrected agreement ($\Delta\kappa = 0.115$), and has the higher agreement in all six projects, although its gain on Closure is not significant. Its advantage is concentrated in the runs that request held-back evidence, and a bug-by-bug study of the transcripts shows that it wins both by retrieving evidence and by forcing the model to state a specific, testable mechanism. About 30% of labels change when the fix becomes visible, with 80% of those changes confined to three closely related types. Running the pipeline with a second model, `gpt-5-mini`, showed that the pre-fix advantage of the loop and the evidence-driven failure modes are properties of the pipeline, while the depth of investigation and the loop’s use of the fix diff depend on the model: `gpt-5-mini` keeps the loop open longer and matches the manual reading for 11 of 13 bugs in post-fix mode, against 6 for Gemini.

7.2 Limitations

In addition to the threats discussed in Chapter 6, the current work has the following limitations:

- **No ground-truth labels:** Defects4J has no ODC annotations, so our evaluation measures reproduction of a better-informed reference rather than correctness.
- **Partial benchmark coverage:** The results cover 410 bugs from six of the 17 projects.
- **Model coverage and single run:** The corpus-level results are produced with one model and the cross-model analysis with two, and each model classifies each bug once per condition, with no consensus mechanism.

7.3 Future Work

Several research directions could extend and improve this work:

1. **Strengthening the scientific loop:** Address the weaknesses identified in Section 5.9: require the loop to investigate before a pre-fix conclusion instead of concluding as early as possible, add the few-shot prompt’s contrastive type guidance to the conclusion step so that a correctly identified mechanism receives the correct label, and require post-fix runs to test the hypothesis against the fix diff.
2. **Adaptive strategy selection:** Make the pipeline choose between the scientific loop and the few-shot prompt for each bug, according to the amount and kind of evidence available, since the loop is the better strategy from symptoms and the few-shot prompt when the fix is visible or the evidence leaves nothing to investigate.
3. **Human reference labels:** Have at least two raters independently label a random sample of bugs from the fix diff, without seeing model output, and use the adjudicated labels to evaluate both evidence modes.
4. **Evaluation of reasoning:** Grade the reasoning trace separately from the label, since the manual analysis shows that the two can diverge.
5. **Multi-model evaluation and consensus:** Apply the cross-model analysis to the full corpus and to additional LLMs, and aggregate classifications from several models to reduce single-model variance [7].
6. **Complete ODC attribute set:** Classify Impact (Section 2.4.1) for bugs whose reports contain sufficient usage detail, derive Trigger and Activity from the same reports using the specification’s mapping [15], and recover Age from project commit history.
7. **Full-dataset and cross-dataset extension:** Complete the remaining Defects4J projects to produce a fully ODC-labelled version of the benchmark, adapt the pipeline to other benchmarks and languages, and use classifications to guide automated program repair [23].

7.4 Closing Remarks

Defect classification is a fundamental building block of software quality assurance, and it has traditionally required expensive manual analysis. This thesis shows that

ODC defect-type classification of Defects4J can be automated at benchmark scale using LLM reasoning, with an escape category that allows the taxonomy's coverage to be measured and an evaluation reference that does not require ground-truth labels. The results indicate that requiring the model to gather and evaluate evidence before answering improves the reliability of its classifications, particularly on projects where a single prompt is least reliable, and that LLM-driven ODC classification is a promising direction for making defect analysis faster, more consistent, and more informative.

References

- [1] J. Agnelo, N. Laranjeiro, and J. Bernardino, “Using Orthogonal Defect Classification to Characterize NoSQL Database Defects,” *Journal of Systems and Software*, vol. 159, p. 110 451, 2020. DOI: 10 . 1016/j . jss . 2019 . 110451
- [2] R. Andrade, C. Teixeira, N. Laranjeiro, and M. Vieira, “An Empirical Study on the Classification of Bug Reports with Machine Learning,” *arXiv preprint*, 2025. arXiv: 2503 . 00660.
- [3] S. Baltes et al., “Guidelines for Empirical Studies in Software Engineering involving Large Language Models,” *Empirical Software Engineering*, 2026, Accepted manuscript. arXiv:2508.15503.
- [4] D. Callaghan and B. Fischer, “Mining Bug Repositories for Multi-Fault Programs,” in *IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), Data and Tool Showcase Track*, arXiv:2403.19171, IEEE, 2025.
- [5] G. Catolino, F. Palomba, A. Zaidman, and F. Ferrucci, “Not All Bugs Are the Same: Understanding, Characterizing, and Classifying Bug Types,” *Journal of Systems and Software*, vol. 152, pp. 165–181, 2019. DOI: 10 . 1016/j . jss . 2019 . 03 . 002
- [6] R. Chillarege et al., “Orthogonal Defect Classification – A Concept for In-Process Measurements,” *IEEE Transactions on Software Engineering*, vol. 18, no. 11, pp. 943–956, 1992. DOI: 10 . 1109/32 . 177364
- [7] G. Colavito, F. Lanubile, N. Novielli, and L. Quaranta, “Large Language Models for Issue Report Classification,” in *Ital-IA 2024 Thematic Workshops, CEUR Workshop Proceedings, Vol. 3762*, 2024, pp. 18–23. [Online]. Available: [https : //ceur-ws . org/Vol-3762/500 . pdf](https://ceur-ws.org/Vol-3762/500.pdf)
- [8] X. Du, Z. Liu, C. Li, X. Ma, Y. Li, and X. Wang, “LLM-BRC: A Large Language Model-Based Bug Report Classification Framework,” *Software Quality Journal*, vol. 32, no. 3, pp. 985–1005, 2024. DOI: 10 . 1007/s11219-024-09675-3
- [9] K. El Emam and I. Wiczorek, “The Repeatability of Code Defect Classifications,” in *Proceedings of the Ninth International Symposium on Software Reli-*

- ability Engineering (ISSRE)*, 1998, pp. 322–333. DOI: 10.1109/ISSRE.1998.730897
- [10] A. M. Esfahani, N. Kahani, and S. A. Ajila, “Understanding Defects in Generated Codes by Language Models,” in *2024 34th International Conference on Collaborative Advances in Software and Computing (CASCON)*, IEEE, 2024, pp. 1–10. DOI: 10.1109/CASCON62161.2024.10837857
 - [11] K. Henningsson and C. Wohlin, “Assuring Fault Classification Agreement – An Empirical Evaluation,” in *Proceedings of the 2004 International Symposium on Empirical Software Engineering (ISESE)*, 2004, pp. 95–104. DOI: 10.1109/ISESE.2004.1334897
 - [12] J. Hernández-González, D. Rodriguez, I. Inza, R. Harrison, and J. A. Lozano, “Learning to Classify Software Defects from Crowds: A Novel Approach,” *Applied Soft Computing*, vol. 62, pp. 579–591, 2018. DOI: 10.1016/j.asoc.2017.10.047
 - [13] T. Hirsch and B. Hofer, “Using Textual Bug Reports to Predict the Fault Category of Software Bugs,” *Array*, vol. 15, p. 100189, 2022. DOI: 10.1016/j.array.2022.100189
 - [14] L. Huang et al., “AutoODC: Automated Generation of Orthogonal Defect Classifications,” *Automated Software Engineering*, vol. 22, no. 1, pp. 3–46, 2015. DOI: 10.1007/s10515-014-0155-1
 - [15] IBM, “Orthogonal Defect Classification v 5.2 for Software Design and Code,” IBM Research, Tech. Rep., 2013.
 - [16] J. Jiang, Y. Xiong, and X. Xia, “A Manual Inspection of Defects4J Bugs and Its Implications for Automatic Program Repair,” *Science China Information Sciences*, vol. 62, no. 10, p. 200102, 2019. DOI: 10.1007/s11432-018-1465-6
 - [17] R. Just, D. Jalali, and M. D. Ernst, “Defects4J: A Database of Existing Faults to Enable Controlled Testing Studies for Java Programs,” in *Proceedings of the International Symposium on Software Testing and Analysis (ISSTA)*, ACM, 2014, pp. 437–440. DOI: 10.1145/2610384.2628055
 - [18] S. Kang, G. An, and S. Yoo, “A Quantitative and Qualitative Evaluation of LLM-Based Explainable Fault Localization,” *Proceedings of the ACM on Software Engineering*, vol. 1, no. FSE, pp. 1424–1446, 2024. DOI: 10.1145/3660771
 - [19] S. Kang, B. Chen, S. Yoo, and J.-G. Lou, “Explainable Automated Debugging via Large Language Model-driven Scientific Debugging,” *Empirical Software Engineering*, vol. 30, no. 2, p. 45, 2025. DOI: 10.1007/s10664-024-10594-x

- [20] A. Koyuncu, “Exploring Fine-Grained Bug Report Categorization with Large Language Models and Prompt Engineering: An Empirical Study,” *ACM Transactions on Software Engineering and Methodology*, vol. 35, no. 3, pp. 1–47, 2026. DOI: 10.1145/3736408
- [21] J. R. Landis and G. G. Koch, “The Measurement of Observer Agreement for Categorical Data,” *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977. DOI: 10.2307/2529310
- [22] C. Li, Z. Zheng, X. Du, X. Ma, Z. Wang, and X. Li, “KnowBug: Enhancing Large Language Models with Bug Report Knowledge for Deep Learning Framework Bug Prediction,” *Knowledge-Based Systems*, vol. 305, p. 112 588, 2024. DOI: 10.1016/j.knosys.2024.112588
- [23] M. Martinez, T. Durieux, R. Sommerard, J. Xuan, and M. Monperrus, “Automatic Repair of Real Bugs in Java: A Large-Scale Experiment on the Defects4J Dataset,” *Empirical Software Engineering*, vol. 22, no. 4, pp. 1936–1964, 2017. DOI: 10.1007/s10664-016-9470-4
- [24] Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai, “Chain-of-Thought Prompting of Large Language Models for Discovering and Fixing Software Vulnerabilities,” *arXiv preprint*, 2024. arXiv: 2402.17230.
- [25] M. N. Rafi, A. R. Chen, T.-H. P. Chen, and S. Wang, “Revisiting Defects4J for Fault Localization in Diverse Development Scenarios,” in *IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR)*, IEEE, 2025, pp. 63–75.
- [26] A. Rahman and E. Farhana, “An Exploratory Characterization of Bugs in COVID-19 Software Projects,” *arXiv preprint*, 2020. arXiv: 2006.00586.
- [27] V. Sobreira, T. Durieux, F. Madeiral, M. Monperrus, and M. d. A. Maia, “Dissection of a Bug Dataset: Anatomy of 395 Patches from Defects4J,” in *IEEE International Conference on Software Analysis, Evolution and Reengineering (SANER)*, IEEE, 2018, pp. 130–140. DOI: 10.1109/SANER.2018.8330203
- [28] F. Thung, D. Lo, and L. Jiang, “Automatic Defect Categorization,” in *Proceedings of the 19th Working Conference on Reverse Engineering (WCRE)*, IEEE, 2012, pp. 205–214. DOI: 10.1109/WCRE.2012.30
- [29] A. Van der Spuy and B. Fischer, “An Anatomy of 488 Faults from Defects4J Based on the Control- and Data-Flow Graph Representations of Programs,” in *Proceedings of the 29th International Conference on Evaluation and Assessment in Software Engineering (EASE)*, ACM, 2025, pp. 622–627. DOI: 10.1145/3756681.3757025